

INFRASTRUKTUR KIDZPLAYFUL — Rencana Penataan & Skala

Dokumen perencanaan infrastruktur dari sudut pandang DevOps. Menjawab satu pertanyaan: **apa yang akan lemot atau tumbang saat pengguna aktif membanyak, dan apa yang harus dikerjakan lebih dulu.**

Disusun: 31 Juli 2026 · **Basis audit:** commit `13d6e17` , migrasi `0001` – `0086` · **Keputusan arah:** tetap di **Vercel + Supabase** (managed, di-scale up) — bukan pindah ke VPS/GCP. **Dokumen pendamping:** `RUNBOOK-OPERASIONAL.md` (prosedur saat kejadian) · `DEVELOPER-KIDZPLAYFUL.md` (arsitektur aplikasi).

0. Ringkasan eksekutif

Audit ini menemukan sesuatu yang perlu disampaikan terang-terangan: **masalah terbesar platform ini bukan kapasitas server.** Vercel dan Supabase akan bertahan lebih lama dari yang diduga. Yang berisiko adalah **pola di dalam aplikasi** — dan tiga di antaranya **sudah merugikan hari ini**, tanpa menunggu pengguna bertambah satu orang pun.

0.1 Tiga hal yang sudah berlaku SEKARANG (bukan risiko masa depan)

#	Temuan	Kondisi nyata	Tindakan	Bab
1	Kredensial admin produksi ada di repo publik	56 kemunculan email + kata sandi <code>admin@kidzplayful.app</code> di 23 skrip <code>tools/*.mjs</code> . Repo publik (syarat Vercel Hobby) → sudah terindeks pencarian kode GitHub. Siapa pun yang menemukannya bisa masuk <code>/admin</code> dan membaca <code>transaksi_keuangan</code> , <code>pendaftaran_event</code> , dan data anak.	Rotasi kata sandi hari ini. Menghapus file tidak cukup — riwayat git tetap menyimpannya.	C.5
2	Angka keuangan & analitik bisa salah tanpa peringatan	<code>keuangan.ts:28</code> , <code>kpi.ts:52-54</code> , dan <code>admin/analitik/page.tsx:29-39</code> menarik tabel penuh tanpa .limit() lalu menjumlahkan di JavaScript. PostgREST memotong hasil di 1.000 baris secara diam-diam — tidak ada error, tidak ada peringatan. Begitu <code>transaksi_keuangan</code> atau <code>profiles</code> melewati 1.000 baris, dashboard menampilkan angka yang salah dengan tenang.	Cek jumlah baris hari ini (D.3.a). Lalu pindahkan agregasi ke SQL.	D.3
3	Bukti bayar & nota keuangan terbuka untuk publik	Bucket <code>aset</code> dideklarasikan <code>public = true</code> dengan policy baca terbuka (<code>0007_storage_aset.sql:2-9</code>). Folder <code>bukti/</code> (bukti transfer orang tua: nama & nomor rekening) dan <code>nota/</code> (nota keuangan internal) berada di URL permanen yang bisa dibuka tanpa login . URL absolutnya tersimpan di DB lalu mengalir ke tangkapan layar, log, dan riwayat browser.	Pindahkan ke bucket privat + signed URL.	F.3

0.2 Dua hal yang baru menyakitkan saat tumbuh

#	Temuan	Kapan pecah	Bab
4	Tidak ada backup sama sekali. Supabase Free tidak menyediakan backup otomatis, dan repo ini tidak punya prosedur <code>pg_dump</code> , tidak punya uji restore, tidak punya dokumentasi DR. Sementara itu 86 migrasi dijalankan manual di SQL Editor produksi — satu <code>delete</code> tanpa <code>where</code> di sana bersifat permanen. Ditambah: backup DB Supabase tidak mencakup file Storage , jadi bukti bayar tidak ter-backup oleh apa pun.	Kapan saja. Ini bukan fungsi dari jumlah pengguna, melainkan dari frekuensi tangan manusia di SQL Editor produksi — dan frekuensi itu tinggi.	C.1
5	<code>catatHasilCore</code> membayar biaya yang tumbuh seumur akun. Setiap kali anak menyelesaikan satu game, <code>skor-core.ts:60</code> menarik seluruh riwayat hasil_main anak itu tanpa <code>.limit()</code> untuk menghitung lencana. Anak yang aktif 6 bulan $\approx$ 540 baris; ditarik ulang setiap sesi. Lebih buruk: begitu satu anak melewati 1.000 baris, hasilnya terpotong diam-diam dan <code>catch {}</code> menelan akibatnya → anak kehilangan lencana & streak tanpa satu pun error .	Sekitar 2.000 pengguna aktif harian (T1) — dan memburuk sendiri walaupun jumlah pengguna tetap, karena yang tumbuh adalah riwayat per anak.	A.3 T1

0.3 Satu keputusan bisnis yang menahan segalanya

Deploy saat ini berjalan di **Vercel Hobby**, dan Hobby **melarang penggunaan komersial**. Platform ini menjual membership, event, dan produk (`store` , `pesanan` , `transaksi_keuangan` , `voucher`) — jadi ini penggunaan komersial. Risikonya bukan tagihan, melainkan **penonaktifan proyek tanpa masa tenggang**.

Naik ke **Vercel Pro + Supabase Pro $\approx$ \$45/bulan** menyelesaikan tiga hal sekaligus: kepatuhan ToS, **backup harian otomatis** (menutup temuan #4), dan repo tidak perlu publik lagi — sehingga seluruh skema dan policy RLS tidak lagi terbuka untuk dibaca siapa pun. Ini item **termurah dengan dampak tertinggi** di seluruh dokumen ini.

0.4 Kabar baiknya

Arsitektur dasarnya sehat dan beberapa keputusan sudah tepat sejak awal, jadi rekomendasi di dokumen ini **melanjutkan arah yang sudah ada**, bukan menggantinya:

- **Region sudah co-located** — Vercel `bom1` (Mumbai) dengan Supabase `ap-south-1`. Ini pemangkas latensi terbesar dan sudah beres.
- **Semua fungsi helper RLS sudah `STABLE SECURITY DEFINER`** dan mencari `profiles` lewat primary key — pilihan yang benar; yang perlu diperbaiki hanya cara memanggilnya di policy (D.2).
- `laporan_engagement()` (`0040_laporan_rpc.sql`) sudah memindahkan agregasi ke SQL dengan guard `is_admin()` — persis pola yang akan direplikasi untuk keuangan & analitik. Presedennya sudah ada di repo ini.
- `lib/data/kuota-event.ts` sudah menerapkan pola akses toleran, lengkap dengan alasannya. Pola itu tetap dipakai, hanya perlu dibatasi perannya (C.4).
- `publik.ts` sudah memberi contoh `select` kolom ringan + `unstable_cache` bertag. Yang kurang hanya penerapannya ke jalur internal (E.2).
- **Service role key terkurung rapi** — satu titik pakai (`lib/supabase/admin.ts`), `import 'server-only'` , tanpa prefix `NEXT_PUBLIC_`. Tidak ada jalur kebocoran ke klien.
- Dependency runtime hanya 6 paket. Tidak ada beban vendor yang perlu dibersihkan.

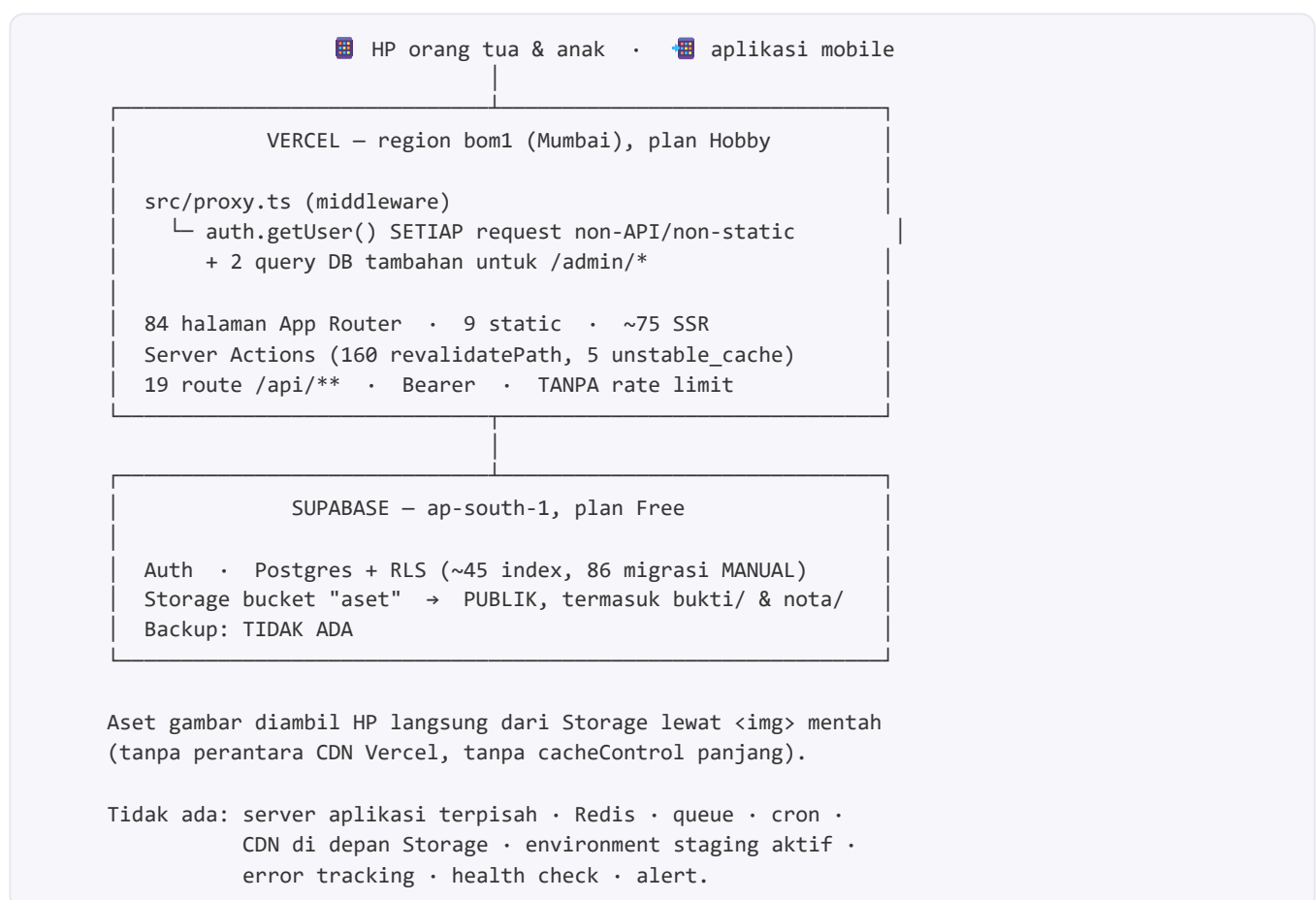
0.5 Ringkasan biaya per tahap

Tahap	Pengguna aktif harian	Biaya/bulan tanpa optimasi	Dengan optimasi di dokumen ini
T0 (sekarang)	< 500	~\$0 (tapi melanggar ToS & tanpa backup)	~\$45
T1	~2.000	~\$85	~\$55–65
T2	~10.000	~\$270	~\$120–150
T3	50.000+	~\$1.900–2.100	~\$700–900

Selisih T2 dan T3 itulah nilai ekonomi dari pekerjaan di [Bab D](#), [E](#), dan [F](#): pada T3 optimasi menghemat lebih dari **\$1.000/bulan**, jauh melampaui biaya kerjanya.

1. Peta kondisi sekarang

1.1 Bentuk sistem



Tidak ada server aplikasi terpisah, tidak ada Redis, tidak ada queue, tidak ada cron, tidak ada CDN di depan Storage, dan tidak ada environment staging yang aktif. "Backend" = Server Components (baca) + Server Actions (tuliskan) + RLS.

1.2 Fakta terverifikasi yang menjadi dasar seluruh rekomendasi

Setiap angka di bawah berasal dari pembacaan kode, bukan perkiraan.

Area	Kondisi	Rujukan
Rendering	0 direktif <code>dynamic</code> / <code>revalidate</code> / <code>generateStaticParams</code> di seluruh repo. Hanya 9 halaman ter-prerender; ~75 dari 84 halaman efektif SSR karena memanggil <code>cookies()</code> .	<code>.next/prerender-manifest.json</code> , <code>lib/supabase/server.ts:6</code>
Caching	Hanya 5 <code>unstable_cache</code> (katalog 60 s, artikel 300 s) melawan 160 <code>revalidatePath</code> di 30 file — invalidasi jauh lebih agresif daripada caching-nya.	<code>lib/data/publik.ts:20,28,36</code> , <code>artikel.ts:70,80</code>
Middleware	<code>auth.getUser()</code> = 1 perjalanan jaringan ke Supabase Auth setiap request non-API/non-static. Matcher tidak mengecualikan <code>.css</code> , <code>.js</code> , <code>.woff2</code> , <code>sitemap.xml</code> , <code>robots.txt</code> , <code>_next/data</code> . Untuk <code>/admin/*</code> ditambah 2 query DB.	<code>src/proxy.ts:26,31,33,53</code>
Halaman terberat	<code>/main/[anakId]</code> ≈ 19–20 perjalanan DB per render, 3 di antaranya <code>auth.getUser()</code> . <code>/admin/event/[id]/pendaftar</code> ≈ 13. <code>/admin/keuangan/laporan</code> memicu 3× full scan identik pada satu render.	<code>app/main/[anakId]/page.tsx:26-56</code> , <code>lib/data/keuangan.ts:107-126</code>
Index	<code>event</code> , <code>produk</code> , <code>tema</code> , <code>aset</code> tidak punya index apa pun selain primary key — padahal <code>status</code> + <code>tanggal</code> / <code>created_at</code> difilter di belasan tempat. <code>pendaftaran_event(status)</code> global memakai index <code>(event_id,status)</code> yang leading column-nya salah.	<code>supabase/migrations/0017,0019,0002,0039</code>
RLS	Fungsi helper sudah <code>STABLE</code> ✓, tapi tidak satu pun policy membungkusnya dalam sub-select → dievaluasi per baris. <code>hasil_main</code> punya 3 policy SELECT permissive yang di-OR , salah satunya <code>exists(... anak ...)</code> per baris.	<code>0002:51</code> , <code>0006:8</code> , <code>0066:27</code>
Payload jsonb	<code>paket_aset.butir</code> (isi game: markup SVG, palette, grid) ikut terkirim untuk semua paket semua tema — termasuk ke <code>/pilih-game</code> yang hanya menampilkan judul & emoji. <code>kelas_bermain.aktivitas</code> + <code>bahan</code> ikut di daftar favorit/riwayat yang juga hanya butuh judul.	<code>lib/data/pustaka.ts:23</code> , <code>favorit.ts:6</code> , <code>riwayat-kelas.ts:5</code>
Bundle klien	98 komponen <code>'use client'</code> . 15 mesin game di-import statis dalam satu berkas, dipilih lewat rantai <code>if/else</code> saat runtime → semuanya masuk chunk rute <code>/main/[anakId]</code> walau anak memainkan satu game. <code>dynamic()</code> / <code>lazy()</code> = 0 pemakaian.	<code>components/game/GameRunner.tsx:5-20,72-86</code>
Gambar	Hanya 7 berkas memakai <code>next/image</code> ; 12 <code></code> mentah di 11 berkas, termasuk <code>Aset.tsx</code> (jalur terpanas) → egress langsung dari Supabase tanpa	<code>components/game/Aset.tsx:10</code> , 12 titik <code>.storage.from(</code>

Area	Kondisi	Rujukan
	perantara. <code>cacheControl</code> tidak pernah diset saat unggah.	
Tabel tumbuh cepat	<code>aktivitas</code> = 1 insert per page view di 9 halaman (dengan 3 index yang ikut diperbarui tiap insert). <code>hasil_main</code> = 1 baris/sesi game. <code>riwayat_kelas</code> = upsert di jalur render GET. <code>transaksi_keuangan</code> tumbuh monoton tanpa arsip, tapi di-full-scan oleh 8 halaman.	<code>components/RekamAktivitas.tsx:11</code> , <code>app/kelas/[id]/page.tsx:33</code>
API	19 route, auth Bearer (1 perjalanan <code>auth.getUser()</code> per request). 0 pemakaian <code>.limit()</code> / <code>.range()</code> di seluruh <code>src/app/api/</code> → bergantung batas diam-diam 1.000 baris. Rate limiting nihil, termasuk <code>/api/auth/login</code> & <code>/register</code> yang publik.	<code>lib/api/helpers.ts:26-40</code>
Observability	Hanya <code>@vercel/analytics</code> . Tanpa Sentry, tanpa <code>instrumentation.ts</code> , tanpa health-check, tanpa alert. <code>app/error.tsx</code> sengaja tidak memakai objek error → error hilang. <code>ledger.ts:35,41</code> membungkus semua tulis keuangan dengan <code>try{}catch{} kosong</code> .	<code>app/error.tsx:2</code> , <code>lib/data/ledger.ts:21-41</code>
CI/CD	Satu workflow (<code>tsc</code> → vitest → build). Tanpa job deploy, tanpa migrasi otomatis, tanpa E2E, tanpa audit dependency, tanpa Dependabot. Branch protection masih berstatus saran.	<code>.github/workflows/ci.yml</code>
Migrasi	86 berkas dijalankan manual, tanpa tabel pelacak. 13 <code>create table</code> dan 8 <code>create index</code> tanpa <code>if not exists</code> ; ~160 <code>create policy</code> sementara hanya 31 berkas memakai <code>drop policy if exists</code> .	<code>supabase/migrations/</code>

1.3 Satu kesimpulan yang paling penting dari tabel di atas

Hari ini tidak ada cara terpercaya untuk membangun ulang produksi dari repo ini. Menjalankan `0001` → `0086` pada database yang sudah terisi sebagian pasti gagal di tengah dan berhenti pada keadaan separuh jalan. Artinya **backup database Anda saat ini lebih bernilai daripada repo Anda** — padahal backup itu belum ada. Membalik keadaan ini adalah inti Bab C.

2. Bab A — Model kapasitas bertingkat

A.0 Catatan pemakaian angka

Semua harga & kuota di bawah adalah **pengetahuan per pertengahan 2026** dan ditandai 🔍 bila **wajib diverifikasi ulang** di halaman pricing sebelum dipakai untuk keputusan anggaran (Vercel & Supabase

mengubah kuota/harga beberapa kali per tahun).

A.1 Asumsi beban (dasar semua hitungan)

Ditulis eksplisit supaya bisa dikoreksi kalau data Vercel Analytics nyata berbeda.

Parameter	Nilai	Dasar
1 DAU	1 akun ortu aktif hari itu	—
Anak per ortu	1,2	—
Page view / DAU / hari	12	1,5 sesi × 8 halaman
Sesi game / DAU / hari	3	tiap sesi memanggil <code>catatHasilCore</code>
Round-trip DB rata-rata / page view	8	terukur: <code>/main/[anakId]</code> = 19–20, <code>/pilih-anak</code> ≈ 4, halaman publik 1–3
Round-trip DB / sesi game	13	<code>src/lib/data/skor-core.ts</code> (5 tahap berurutan)
Insert <code>aktivitas</code> / DAU / hari	4	9 halaman ber- <code>RekamAktivitas</code>
Payload aset Supabase efektif / PV	0,4 MB	<code></code> mentah, tanpa <code>next/image</code>
Payload HTML+JS / PV	0,3 MB	75 halaman SSR
Konsentrasi puncak	40% trafik dalam 3 jam	jam sore/malam anak

Bukti hitungan 19–20 round-trip pada `/main/[anakId]` (rantai nyata, bukan asumsi): `proxy.ts`
`auth.getUser()` (1) → `getAnakTerjamin` : `auth.getUser()` (2) + `anak` (3) + `langganan` (4) → `createClient` + `auth.getUser()` (5) → paralel: `getVideoByKategori` (6), `getPustaka` = `tema` + `paket_aset` + `video` (7–9), `getKelasAktifCached` (10, cached 60s), `getFavoritIds` (11–12), `profiles.pin_ortu` (13), `getGamifikasiAnak` = 5 query (14–18), `getStatusLangganan` (19), `getLabelFokusArea` (20). → **3 kali `auth.getUser()` untuk satu render halaman.**

A.2 Tabel beban unit (per 1.000 DAU)

Metrik	/ hari	/ bulan (30 hari)	Puncak (rata 3 jam sibuk)
Page view	12.000	360.000	1,1 PV/detik
Request Vercel total (PV + aset + Server Action + API)	~45.000	1,35 juta	~4 req/detik
Function invocation (non-static)	~15.000	450.000	~1,4 /detik
Hit Supabase Auth (<code>auth.getUser</code>)	~14.400	432.000	~1,3 /detik
Query DB total	~140.000	4,2 juta	~13 query/detik
Insert <code>aktivitas</code>	4.000	120.000	—
Insert <code>hasil_main</code>	3.600	108.000	—
Egress Supabase Storage	4,8 GB	144 GB	—
Vercel Fast Data Transfer	3,6 GB	108 GB	—

Rincian 140.000 query/hari: `12.000 PV × 8` = 96.000 + `3.000 sesi × 13` = 39.000 + `aktivitas` 4.000 + `riwayat_kelas` upsert ~1.000.

A.3 Empat tier

T0 — SEKARANG (< 500 DAU)

Beban (0,5× unit): 6.000 PV/hari · 70.000 query/hari (2,1 jt/bln) · 225k invocation/bln · **egress Supabase ~72 GB/bln** · FDT ~54 GB/bln · puncak ~7 query/detik.

Plan & biaya:

Item	Sekarang	Seharusnya	USD/bln
Vercel	Hobby	Pro (wajib, lihat failure mode)	20 🔍
Supabase	Free	Pro (Micro tercakup credit \$10)	25 🔍
Egress Supabase	—	72 GB vs 250 GB termasuk	0
Total	~\$0		~\$45

Batas plan yang relevan (🔍 semua):

	Vercel Hobby	Vercel Pro	Supabase Free	Supabase Pro
Fast Data Transfer	100 GB	1 TB (over ~\$0,15/GB)	—	—
Function compute	100 GB-jam	1.000 GB-jam (over ~\$0,18/GB-jam)	—	—
Edge requests	1 juta	10 juta (over ~\$2/juta)	—	—
maxDuration default	~10 s	~10 s (bisa dinaikkan s.d. 300 s)	—	—
Cron	2, hanya harian	40, jadwal bebas	—	—
Ukuran DB	—	—	500 MB	8 GB termasuk
Egress	—	—	5 GB	250 GB (over ~\$0,09/GB)
Storage file	—	—	1 GB	100 GB
MAU Auth	—	—	50.000	100.000
Backup	—	—	TIDAK ADA	daily, retensi 7 hari
PITR	—	—	tidak	add-on ~\$100/bln
Pause otomatis	—	—	setelah 7 hari idle	tidak
Koneksi (Micro/Small)	—	—	~60 direct / 200 pooler	200→400 pooler
Penggunaan komersial	DILARANG (ToS)	boleh	boleh	boleh

Catatan arsitektur penting: aplikasi memakai `supabase-js` → PostgREST via HTTPS, **bukan koneksi Postgres langsung**. Jadi metrik "connections" di dashboard bukan bottleneck utama; yang mengikat adalah **CPU Supabase + Disk IO budget + saturasi worker PostgREST**. Koneksi langsung hanya relevan untuk `pg_dump` dan migrasi.

Pekerjaan yang HARUS selesai di T0 (sebelum promosi apa pun):

#	Pekerjaan	Effort	Prioritas
1	Rotasi password <code>admin@kidzplayful.app</code> + audit jejak masuk	1 jam	P0
2	Kredensial <code>tools/</code> → env var + guard anti-produksi	3 jam	P0
3	Rate limit <code>/api/auth/login</code> & <code>/register</code>	2 jam	P0
4	<code>file_size_limit</code> + <code>allowed_mime_types</code> di bucket <code>aset</code>	15 menit	P0
5	<code>.limit()</code> / <code>.range()</code> di 19 route handler	4–6 jam	P0
6	Sentry + <code>instrumentation.ts</code> + <code>error.tsx</code> melaporkan	3 jam	P0
7	<code>/api/health</code> + <code>/api/health/db</code> + uptime monitor	2 jam	P0
8	<code>schema_migrations</code> + <code>0000_baseline.sql</code> + runner	6 jam	P0
9	<code>pg_dump</code> script + uji restore pertama	4 jam	P0
10	<code>ledger.ts</code> anti-telan + tabel <code>ledger_gagal</code>	4 jam	P0
11	Pisah bucket <code>privat</code> untuk <code>bukti/</code> & <code>nota/</code> (lihat F.3)	1–1,5 hari	P0 (≤ 2 minggu)
12	Branch protection <code>master</code>	15 menit	P0

Yang PECAH lebih dulu bila tidak dikerjakan — di T0, hari ini:

- Kredensial admin produksi di repo publik.** 56 kemunculan di 23 skrip, sudah terindeks GitHub code search. Siapa pun bisa login `/admin` sekarang → akses `transaksi_keuangan`, `pendaftaran_event`, data anak. Ini **bukan risiko masa depan, ini kondisi saat ini**.
- Bukti bayar ortu & nota keuangan di URL publik permanen.** `0007_storage_aset.sql`: `create policy "aset baca publik" ... for select using (bucket_id = 'aset') + public = true` → `/storage/v1/object/public/aset/bukti/<Date.now().jpg` bisa dibrute (nama berbasis timestamp = ruang tebakan kecil). Data pribadi keluarga + bukti transfer bank.
- Egress Supabase Free 5 GB vs kebutuhan ~72 GB/bln.** Sudah pecah pada 500 DAU. Gejalanya bukan error jelas, tapi **throttling / project restriction** — game tampak "gambar tidak muncul".
- No! backup + 86 migrasi manual di SQL Editor produksi.** Satu `delete` / `drop` salah tanpa `where` = kehilangan permanen. Frekuensi tangan owner di SQL Editor produksi tinggi (86 file dijalankan manual) → probabilitas kejadian tidak kecil.
- Vercel Hobby melarang penggunaan komersial.** Ada `store`, `pesanan`, `transaksi_keuangan`, `voucher` → ini komersial. Risiko suspensi mendadak tanpa masa tenggang.
- PostgREST 1000 baris tanpa `.limit()`.** Sudah aktif sekarang bila ada tabel > 1000 baris: laporan keuangan/aktivitas **terpotong silent** → angka di dashboard investor salah tanpa satu pun error.

T1 — ~2.000 DAU

Beban (2× unit): 24.000 PV/hari · **280.000 query/hari (8,4 jt/bln)** · 900k invocation/bln · egress ~288 GB/bln · FDT ~216 GB/bln · puncak ~2,2 PV/detik → **~26 query/detik**.

Pemicu naik ke T1 (naik bila **salah satu** terpenuhi 7 hari berturut-turut):

Metrik	Ambang	Di mana dilihat
p95 TTFB <code>/main/[anakId]</code>	> 1.500 ms	Vercel → Observability → per-route p95
CPU Supabase rata-rata harian	> 40%	Supabase → Reports → Database
CPU spike > 80%	> 5 menit/hari	idem
Disk IO budget tersisa	< 80%	Supabase → Reports → Database → Disk IO
Ukuran DB	> 400 MB (limit Free 500 MB)	Supabase → Settings → Usage
Egress Supabase	> 4 GB/bln (Free) atau > 200 GB (Pro)	Supabase → Usage
Function invocation Vercel	> 700k/bln	Vercel → Usage
Vercel GB-jam	> 80/bln (limit Hobby 100)	Vercel → Usage

Plan & biaya T1:

Item	USD/bln
Vercel Pro	20 🔍
Supabase Pro	25 🔍
Compute Small (2 GB RAM) — net setelah credit \$10	+5...15 🔍
Egress 288 GB → 38 GB over × \$0,09	3,4
Total	~\$55–65 (tanpa optimasi egress bisa \$85)

Pekerjaan yang HARUS selesai SEBELUM masuk T1:

#	Pekerjaan	Dampak	Effort	Prasyarat	Prio
1	<code>catatHasilCore</code> berhenti menarik seluruh <code>hasil_main</code> → kolom agregat di <code>anak</code> (<code>total_selesai</code> , <code>mesin_mask</code> , <code>ada_bintang3</code>) atau satu RPC <code>catat_hasil_main()</code> SECURITY DEFINER; 13 op → 1–2	Hilangkan pertumbuhan biaya seumur akun	1–2 hari	migrasi + backfill	P1 (blocker T1)
2a	<code>proxy.ts</code> : matcher kecualikan <code>.css/.js/.woff2/sitemap.xml/robots.txt/_next/data</code>	Hentikan invocation + <code>auth.getUser()</code> untuk sub-resource	15 menit	—	P0
2b	<code>auth.getUser()</code> → <code>auth.getClaims()</code> (verifikasi tanda tangan lokal , bukan membaca token tanpa verifikasi — lihat §9 butir 4)	–1 round-trip Auth per PV (–432k/bln per 1.000 DAU) + –50...150 ms TTFB	3–4 jam	Asymmetric JWT signing keys aktif 🔍	P1
3	Hilangkan <code>auth.getUser()</code> ganda di <code>/main</code> (pakai 1× lalu teruskan <code>user</code>)	–2 dari 20 round-trip	2 jam	—	P1
4	<code>aktivitas</code> : sampling (1:5) atau batch per sesi; turunkan 3 index → 1 komposit	Kurangi write amplification 3×	3 jam	—	P1
5	<code>riwayat_kelas</code> upsert keluar dari jalur GET → Server Action fire-and-forget dari client	Hilangkan WRITE saat prefetch/crawler	2 jam	—	P1
6	<code>ChatKonsultasi.tsx:66</code> : polling 3 s → <code>?after=<id></code> + <code>.limit(50)</code> , interval 5 s + backoff saat tab tidak fokus	20 req/menit/ortu × N	3 jam	—	P1
7	Aset game: <code>cacheControl</code> setahun + preset 256 px (BUKAN <code>next/image</code> — lihat E.4 & §9)	Egress Supabase turun ~20×	1,5 jam	—	P1
8	Strategi cache: <code>revalidate</code> eksplisit + perluas <code>unstable_cache</code> ; ganti sebagian dari 160 <code>revalidatePath</code> ke tag	Turunkan SSR & query/PV	2–3 hari	—	P1
9	<code>vercel.json</code> : <code>functions.maxDuration</code> per grup route	Query lari kencang gagal cepat, bukan bakar GB-jam	30 menit	Pro	P1
10	Load test k6 pada <code>/main/[anakId]</code> + <code>POST /api/hasil-main</code>	Validasi angka tabel A.2	1 hari	staging/beta	P1

Yang PECAH lebih dulu di T1 bila tidak dikerjakan:

1. `catatHasilCore` — ini yang pertama tumbang.

`supabase.from('hasil_main').select('mesin,bintang,tanggal,tema_id,paket_id').eq('anak_id', ...)`

tanpa limit. Anak aktif 6 bulan = 3 sesi × 180 hari ≈ **540 baris**. Pada 2.000 DAU = 6.000 sesi/hari → **3,2 juta baris/hari ditarik hanya untuk menghitung lencana**. Konsekuensi ganda: (a) CPU + Disk IO Supabase, (b) begitu satu anak melewati **1000 baris**, PostgREST memotong silent → `evaluasiLencana`

salah hitung dan `catch{}` di baris 118 menelan semuanya → **anak kehilangan lencana & streak tanpa error apa pun**. Biaya per sesi tumbuh seumur akun; artinya masalah ini memburuk walaupun DAU tetap.

2. `/api/auth/login` & `/register` **tanpa rate limit** + password admin publik. Pada 2.000 DAU, satu bot 50 req/detik = 4,3 juta percobaan/hari: menghabiskan kuota Auth Supabase, memicu throttle yang **mengunci login ortu asli**, dan membakar function invocation Vercel.
3. **Batas 1000 baris di 19 route API**. Pada T1 `produk`, `event`, `pesanan`, `transaksi_keuangan` melewati 1000 → aplikasi mobile Flutter menerima data terpotong, halaman keuangan menampilkan total yang salah. **Tidak ada error, tidak ada alert**.
4. **75 halaman SSR tanpa `revalidate`**. 900k invocation/bln × durasi rata-rata melar karena p95 DB naik → GB-jam Vercel mendaki lebih cepat dari trafik.
5. `aktivitas` **120k insert/bln per 1.000 DAU = 240k/bln di T1** dengan 3 index yang di-update tiap insert, di jalur PAGE VIEW, fire-and-forget → kegagalan tak terlihat, tapi biaya IO-nya tetap dibayar.

T2 — ~10.000 DAU

Beban (10× unit): 120.000 PV/hari · **1,4 juta query/hari (42 jt/bln)** · 4,5 jt invocation/bln · egress Supabase ~1,44 TB/bln · FDT ~1,08 TB/bln · puncak ~11 PV/detik → **~90 query/detik**.

Pemicu naik ke T2:

Metrik	Ambang
CPU Supabase p95 (di Small)	> 60%
Egress Supabase	> 200 GB/bln (menuju 250 GB termasuk)
Ukuran DB	> 6 GB (dari 8 GB termasuk)
p95 TTFB seluruh site	> 1.000 ms
Pooler connection utilization	> 60%
Error rate (Sentry / Vercel)	> 1% request
Vercel FDT	> 800 GB/bln
Vercel GB-jam	> 800/bln

Plan & biaya T2 (dua skenario — argumen kuat untuk mengerjakan optimasi T1 dulu):

Item	Tanpa optimasi	Dengan optimasi Bab E + F
Vercel Pro	20	20
Observability Plus	10	10
FDT over 1 TB	~\$12	~\$0
Fluid compute GB-jam over	~\$45	~\$0
Supabase Pro	25	25
Compute Medium (4 GB) net	+50 🔍	+50 🔍
Egress Supabase over 250 GB	~\$107	~\$16
Total	~\$270/bln	~\$120-150/bln

Pekerjaan wajib sebelum T2:

#	Pekerjaan	Effort	Prio
1	Arsip/partisi bulanan aktivitas + hasil_main (cron Vercel + tabel *_arsip)	2-3 hari	P2
2	transaksi_keuangan : tabel rollup/materialized view bulanan di atas RPC dari D.3 (RPC-nya sendiri sudah P0, bukan P2)	1 hari	P2
3	Rate limit terdistribusi (Vercel Firewall atau Upstash)	1 hari	P2
4	CDN (Cloudflare gratis) di depan Supabase Storage / pindahkan aset statis game ke public/ Vercel	1 hari	P2
5	Aktifkan PITR (~\$100/bln 🔍)	1 jam	P2
6	Environment beta/staging identik + uji migrasi di sana lebih dulu	1 hari	P1
7	Read replica untuk halaman laporan/investor (butuh compute ≥ Small 🔍)	1 hari	P2

Yang pecah di T2: halaman admin keuangan **timeout** (8 halaman full-scan **transaksi_keuangan** yang tumbuh monoton tanpa arsip, dengan **maxDuration** default ~10 s); **aktivitas** ≈ 1,2 juta baris/bulan membuat halaman analitik tak bisa dibuka; biaya egress naik ke ~\$107/bln hanya karena aset tanpa cache panjang dan berukuran 3-4× lebih besar dari yang pernah terlihat; ChatKonsultasi polling 3 s × ratusan ortu jadi sumber trafik konstan terbesar.

T3 — 50.000+ DAU

Beban (50× unit): 600.000 PV/hari · **7 juta query/hari (210 jt/bln)** · 22,5 jt invocation/bln · egress ~7,2 TB/bln · FDT ~5,4 TB/bln · puncak ~55 PV/detik → **~450 query/detik**.

Pemicu: CPU > 70% di Medium · insert **hasil_main** > 50/detik · **aktivitas** > 50 juta baris · egress > 2 TB/bln · p95 write **POST /api/hasil-main** > 800 ms.

Biaya: tanpa optimasi **~\$1.900-2.100/bln** (FDT over ~\$660, compute over ~\$270, egress Supabase ~\$625, XL compute, PITR, read replica). Dengan CDN + cache agresif + partisi: **~\$700-900/bln**. Pada level ini masuk akal negosiasi Vercel Team/Enterprise dan Supabase komitmen tahunan 🔍.

Wajib sebelum T3: partisi tabel deklaratif (`hasil_main` , `aktivitas` per bulan); **queue** (tabel job + Vercel Cron, atau QStash) untuk gamifikasi/notifikasi keluar dari jalur request; read replica untuk semua baca laporan; staging identik + smoke test otomatis; RPO $\leq$ 5 menit. **Yang pecah tanpa itu:** satu primary Postgres jadi titik tunggal untuk 450 query/detik campur write-heavy $\rightarrow$ lock contention di `anak.koin` update (setiap sesi game meng-update baris `anak` yang sama untuk anak yang sama; dengan 3 sesi/hari tak bermasalah, tapi jalur `update anak set koin` + `upsert lencana_anak` dalam satu `Promise.all` tanpa transaksi = **race koin** yang di T3 pasti muncul).

3. Bab B — Ketersediaan & observability

B.1 Health-check: dua lapis (jangan satu endpoint gemuk)

Prinsip: endpoint yang dipanggil tiap menit oleh monitor **tidak boleh menyentuh DB**; endpoint yang menyentuh DB **harus dilindungi** agar tidak jadi amplifikasi serangan.

Yang DICEK: proses hidup, commit SHA, region, satu `HEAD count` ke tabel 1-baris (`pengaturan_menu`), versi migrasi terpasang. **Yang TIDAK BOLEH dicek:** `auth.getUser()` (round-trip Auth = vektor beban ke kuota Auth), `storage.list()` , RPC `laporan_engagement()` , tabel besar (`transaksi_keuangan` / `hasil_main` / `aktivitas`), dan **tidak boleh ada satu pun operasi TULIS** (jangan sekali-kali memanggil `catatAktivitas` di sini).

Artefak: `src/app/api/health/route.ts`

```
// src/app/api/health/route.ts – LIVENESS. Wajib murah: tanpa DB, tanpa Auth, tanpa Storage.
// Dipanggil uptime monitor tiap 60 detik. Aman dipublikasikan.
export const dynamic = 'force-dynamic';
export const runtime = 'nodejs';

export async function GET() {
  return Response.json(
    {
      ok: true,
      layanan: 'kidzplayful-web',
      commit: process.env.VERCEL_GIT_COMMIT_SHA?.slice(0, 7) ?? 'lokal',
      env: process.env.VERCEL_ENV ?? 'development',
      region: process.env.VERCEL_REGION ?? 'lokal',
      waktu: new Date().toISOString(),
    },
    { status: 200, headers: { 'cache-control': 'no-store' } },
  );
}
```

Artefak: `src/app/api/health/db/route.ts`

```

// src/app/api/health/db/route.ts – READINESS. Dilindungi header rahasia (balas 404 bila salah,
// bukan 401, supaya tidak mengumumkan keberadaannya). Dipanggil monitor tiap 5 menit.
//
// DICEK      : (1) DB terjangkau via HEAD count tabel 1-baris, (2) versi skema >= yang dibutuhkan
kode.
// TIDAK DICEK: auth.getUser(), storage, RPC laporan, tabel besar. TIDAK ADA operasi tulis.
import { createClient } from '@supabase/supabase-js';

export const dynamic = 'force-dynamic';
export const runtime = 'nodejs';
export const maxDuration = 10;

const BATAS_MS = 2000;
const MIGRASI_MINIMAL = process.env.MIGRASI_MINIMAL ?? '0086';

export async function GET(req: Request) {
  if (req.headers.get('x-health-key') !== process.env.HEALTH_KEY) {
    return new Response('Not Found', { status: 404 });
  }

  const s = createClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.SUPABASE_SERVICE_ROLE_KEY!,      // perlu utk baca schema_migrations (RLS tertutup)
    { auth: { persistSession: false } },
  );

  const t0 = Date.now();
  const cek: Record<string, unknown> = {};
  let sehat = true;

  // (1) DB terjangkau – tabel konfigurasi 1 baris, HEAD saja (tanpa transfer data)
  const db = (await Promise.race([
    s.from('pengaturan_menu').select('id', { count: 'exact', head: true }).eq('id', 1),
    new Promise((r) => setTimeout(() => r({ error: { message: `timeout > ${BATAS_MS}ms` } })),
    BATAS_MS)),
  ])) as { error?: { message: string } | null };
  cek.db = db?.error ? { ok: false, pesan: db?.error?.message } : { ok: true };
  if (db?.error) sehat = false;

  // (2) Deteksi "kode mendahului migrasi" – insiden kuota_* tidak boleh terulang tanpa terdeteksi
  const { data: mig } = await s
    .from('schema_migrations').select('versi').order('versi', { ascending: false }).limit(1);
  const versiDb = mig?.[0]?.versi ?? '0000';
  const migrasiOk = versiDb >= MIGRASI_MINIMAL;
  cek.migrasi = { versi_db: versiDb, minimal_kode: MIGRASI_MINIMAL, ok: migrasiOk };
  if (!migrasiOk) sehat = false;

  return Response.json(
    { ok: sehat, durasi_ms: Date.now() - t0, commit: process.env.VERCEL_GIT_COMMIT_SHA?.slice(0, 7),
    cek },
    { status: sehat ? 200 : 503, headers: { 'cache-control': 'no-store' } },
  );
}

```

Dampak: deteksi mati total < 2 menit; deteksi race kode-vs-migrasi < 5 menit (insiden `kuota_*` akan tertangkap otomatis). **Effort 2 jam. Prasyarat:** tabel `schema_migrations` (Bab C), env `HEALTH_KEY`. **Prioritas P0.**

B.2 Error tracking: perbandingan singkat & keputusan

Opsi	Free tier 🔍	Plus	Minus untuk kasus ini
Sentry	5.000 error/bln, ~10k span, retensi 30 hari, 1 seat	SDK Next.js 16 resmi (<code>instrumentation.ts</code> + <code>onRequestError</code>), menangkap Server Component / Server Action / Route Handler / proxy; source map otomatis; integrasi Vercel set env sendiri	+30–40 KB bundle client; kuota free habis cepat kalau noise tak difilter
Vercel Log Drain → Axiom	~500 MB/bln	Zero SDK, satu tempat dengan runtime log	Bukan error tracker: tak ada grouping/dedup/stack-trace-aware; alert manual
Highlight.io	murah hati	session replay bagus	ekosistem lebih kecil, dokumen Next 16 tertinggal
GlitchTip (self-host)	gratis	API-compatible Sentry	butuh server + backup → menambah 1 sistem yang harus dijaga; anti-tujuan untuk tim 1 orang

Keputusan: Sentry (Developer/free). Alasan spesifik: satu-satunya yang punya `onRequestError` sehingga error yang saat ini **hilang total** (`src/app/error.tsx` sengaja tidak memakai objek `error`) langsung dilaporkan tanpa mengubah 84 halaman satu-satu. Session Replay & Profiling **dimatikan** di awal untuk menghemat kuota. `tracesSampleRate: 0.05` .

Artefak: `instrumentation.ts` (root proyek, sejajar `next.config.ts`)

```
// instrumentation.ts – dijalankan Next.js sebelum kode aplikasi, di semua runtime.
import * as Sentry from '@sentry/nextjs';

export async function register() {
  if (process.env.NEXT_RUNTIME === 'nodejs') await import('./sentry.server.config');
  if (process.env.NEXT_RUNTIME === 'edge') await import('./sentry.edge.config'); // src/proxy.ts
}

// Hook Next.js: menangkap error dari Server Component, Server Action, Route Handler, dan proxy.
// Ini yang menggantikan "error hilang" di src/app/error.tsx.
export const onRequestError = Sentry.captureRequestError;
```

Artefak: `sentry.server.config.ts`

```
import * as Sentry from '@sentry/nextjs';

Sentry.init({
  dsn: process.env.SENTRY_DSN,
  environment: process.env.VERCEL_ENV ?? 'development',
  release: process.env.VERCEL_GIT_COMMIT_SHA,
  tracesSampleRate: 0.05, // hemat kuota free tier
  profilesSampleRate: 0,
  sendDefaultPii: false, // WAJIB: aplikasi anak
  ignoreErrors: ['NEXT_REDIRECT', 'NEXT_NOT_FOUND'], // redirect() bukan error
  beforeSend(ev) {
    delete ev.request?.cookies; // cookie Supabase = token sesi
    if (ev.request?.headers) { delete ev.request.headers.cookie; delete
ev.request.headers.authorization; }
    return ev;
  },
});
```

Artefak: `src/app/error.tsx` (perbaikan minimal)

```
'use client';
import { useEffect } from 'react';
import * as Sentry from '@sentry/nextjs';

export default function Error({ error, reset }: { error: Error & { digest?: string }; reset: () =>
void }) {
  useEffect(() => { Sentry.captureException(error); }, [error]);
  return (
    <main /* ...gaya lama tetap... */>
      <div style={{ fontSize: 60 }}>🤖</div>
      <h1>Yah, ada sedikit gangguan</h1>
      <p>Coba lagi sebentar ya.</p>
      <button className="kp-btn" onClick={reset}>Coba lagi</button>
      {error.digest && <small style={{ opacity: .5 }}>kode: {error.digest}</small>}
    </main>
  );
}
```

Dampak: dari 0% error terlihat → ~95% terlihat. **Effort 3 jam** (termasuk wizard `npx @sentry/wizard -i nextjs`). **Prasyarat:** akun Sentry, env `SENTRY_DSN` + `SENTRY_AUTH_TOKEN` (source map). **P0.**

B.3 Logging terstruktur minimal (cocok untuk 1 orang)

Aturan: **satu util, satu baris JSON per event, tanpa dependensi.** Vercel Runtime Logs mem-parse JSON otomatis; retensi lebih dari 1 jam butuh Log Drain (Pro) → arahkan ke Axiom free 🗨️ hanya bila sudah terasa perlu (P2).

Artefak: `src/lib/obs/log.ts`

```
// src/lib/obs/log.ts – log terstruktur. SATU baris = SATU JSON.
// ATURAN PRIVASI (wajib): DILARANG menulis nama anak, tanggal_lahir, email ortu, nomor WA,
// URL bukti bayar. Pakai identitas ter-hash: ortu_hash / anak_hash.
import { createHash } from 'node:crypto';

type Level = 'info' | 'warn' | 'error';

export function hashId(id?: string | null) {
  return id ? createHash('sha256').update(id).digest('hex').slice(0, 12) : null;
}

export function log(level: Level, event: string, fields: Record<string, unknown> = {}) {
  const baris = JSON.stringify({
    ts: new Date().toISOString(),
    level,
    event, // contoh: 'ledger.gagal', 'skor.catat',
    'auth.rate_limit'
    commit: process.env.VERCEL_GIT_COMMIT_SHA?.slice(0, 7),
    env: process.env.VERCEL_ENV ?? 'dev',
    ...fields,
  });
  if (level === 'error') console.error(baris);
  else if (level === 'warn') console.warn(baris);
  else console.log(baris);
}
```

Daftar event minimal yang wajib di-log (10 event, bukan 100): `auth.login_gagal` · `auth.rate_limit` · `ledger.gagal` · `skor.catat_fallback` (cabang `catch` di `skor-core.ts`) · `migrasi.tertinggal` · `db.error` · `storage.unggah_gagal` · `postgrest.terpotong` (saat hasil = tepat 1000 baris → tanda pemotongan silent) · `kuota.hampir_habis` · `cron.selesai` .

`postgrest.terpotong` adalah alat murah yang langsung mengubah kegagalan silent menjadi terlihat: bungkus reader panas dengan cek `if (data?.length === 1000) log('warn', 'postgrest.terpotong', {tabel})` . **Effort 1 jam, dampak besar. P0.**

Effort util + penyisipan 10 event: 4 jam. P0.

B.4 Alert yang layak membangunkan orang (maksimal 6)

Pisahkan tegas **PAGE** (bunyi, WhatsApp/telepon) dari **DIGEST** (email harian, tidak membangunkan).

#	Alert	Ambang	Kanal	Mengapa layak (kait ke audit)
1	Situs mati	<code>/api/health</code> gagal 2× berturut (interval 60 s)	PAGE	tak ada monitor sama sekali sekarang
2	Kegagalan tulis keuangan	<code>event=ledger.gagal</code> ≥ 1 kejadian	PAGE	<code>ledger.ts</code> saat ini <code>catch {}</code> kosong → uang hilang dari catatan tanpa jejak
3	Lonjakan error	error rate > 2% request selama 5 menit ATAU > 25 event error / 5 menit	PAGE	mayoritas kegagalan Supabase ditelan; tanpa ini, rilis rusak baru diketahui dari keluhan ortu
4	Skema tertinggal dari kode	<code>/api/health/db</code> balas 503 dengan <code>cek.migrasi.ok=false</code>	PAGE	mengulang insiden <code>kuota_*</code> (daftar event kosong, user gagal daftar)
5	Supabase kritis	CPU > 85% selama 10 menit ATAU disk > 85% ATAU Disk IO budget < 20%	PAGE	Micro/Small punya IO burst budget yang bisa habis → seluruh DB melambat
6	Serangan auth	> 100 <code>auth.login_gagal</code> / 5 menit dari 1 IP, atau > 500 total / 5 menit	PAGE	<code>/api/auth/*</code> tanpa rate limit dan password admin ada di repo publik

Digest (email harian, TIDAK page): p95 TTFB per route, jumlah `postgrest.terpotong`, proyeksi egress & GB-jam bulan ini, pertumbuhan ukuran DB & bucket, jumlah baris `ledger_gagal` yang belum di-replay, jumlah `skor.catat_fallback`.

Tambahan wajib ke digest — pelajaran dari insiden 5 Agustus 2026: bandingkan **commit terakhir di Git** dengan **commit yang benar-benar tersaji di produksi**. Deploy pernah berhenti diam-diam selama ~5 hari (repo privat + plan Hobby) tanpa memicu satu pun sinyal: push sukses, CI hijau, tidak ada error. Ini **bukan** alert yang layak membangunkan orang — cukup satu baris di email harian, tapi tanpa itu waktu deteksinya adalah *hari*, bukan jam. Cara memeriksanya ada di [RB-10](#).

Artefak: konfigurasi alert (Sentry — Alerts → Create Alert)

```
# Alert 2 – Kegagalan tulis keuangan (paling ketat, ambang 1)
nama: "P0 Keuangan: ledger gagal"
tipe: Issue Alert
filter:
  - "message" contains "ledger.gagal"      # atau tags: area = keuangan
kondisi:
  - "Jumlah event" >= 1 dalam 1 menit
aksi:
  - Notifikasi: WhatsApp via webhook / email owner
  - Prioritas: critical
frekuensi_ulang: 5 menit

# Alert 3 – Lonjakan error
nama: "P0 Lonjakan error produksi"
tipe: Metric Alert
metrik: failure_rate() # events dengan level error / total
filter: environment:production
kondisi:
  - kritis: "> 2%" selama 5 menit
  - peringatan: "> 1%" selama 10 menit
aksi: kritis -> WhatsApp; peringatan -> email
```

```
# Alert 1 & 4 – Better Stack / UptimeRobot (free tier 🔍)
monitor_liveness:
  url: https://www.kidzplayful.com/api/health
  interval: 60s
  gagal_setelah: 2 kali
  wilayah: [Mumbai, Singapore]
  eskalasi: WhatsApp + telepon setelah 3 menit
monitor_readiness:
  url: https://www.kidzplayful.com/api/health/db
  header: { x-health-key: "<HEALTH_KEY>" }
  interval: 300s
  harapkan_status: 200
  gagal_setelah: 2 kali
```

Alert 5 (Supabase): notifikasi bawaan Supabase terbatas 🔍 → paling andal adalah **Vercel Cron harian** memanggil Management API lalu kirim notifikasi:

```
// vercel.json (versi target)
{
  "$schema": "https://openapi.vercel.sh/vercel.json",
  "regions": ["bom1"],
  "functions": {
    "src/app/api/**/route.ts": { "maxDuration": 15 },
    "src/app/admin/keuangan/**": { "maxDuration": 60 }
  },
  "crons": [
    { "path": "/api/cron/cek-kuota", "schedule": "0 1 * * *" },
    { "path": "/api/cron/arsip-aktivitas", "schedule": "30 18 * * *" }
  ]
}
```

Route cron dilindungi `Authorization: Bearer ${CRON_SECRET}`. **Cron sub-harian & > 2 cron butuh Vercel Pro** 🔍.

Effort total B.4: 4 jam. Prasyarat: Sentry + health endpoint. **P0** (alert 1–4), **P1** (alert 5–6).

B.5 Rate limiting: penempatan tanpa Redis

Opsi	Cara kerja	Plus	Minus	Cocok untuk
A. Vercel Firewall (rate limit rule)	di edge, sebelum function	tidak memakan invocation sama sekali; persisten lintas instance; bisa diubah tanpa deploy; bisa CAPTCHA/challenge	butuh Pro 🔍; dikonfigurasi di dashboard (tidak versioned di git)	<code>/api/auth/*</code> , <code>/login</code> , <code>/register</code> — pilihan akhir
B. In-memory di route (Map + sliding window)	per instance function	gratis, 0 dependensi, 30 menit kerja	reset tiap cold start ; tiap instance punya hitungan sendiri → serangan terdistribusi lolos; serverless bisa punya puluhan instance	stop-gap hari ini + lapisan kedua permanen
C. Upstash Redis (@upstash/ratelimit)	KV global	akurat lintas instance & region; kuota per-user, bukan hanya per-IP; free ~10k command/hari 🔍	+1 hop jaringan (~10–30 ms dari <code>bom1</code> , pilih region terdekat Mumbai); +1 vendor & 1 lagi yang bisa mati	T2, saat butuh kuota per-user (mis. <code>POST /api/hasil-main</code>)

Keputusan bertingkat: - **Sekarang (P0, 2 jam):** Opsi **B** di `/api/auth/login`, `/api/auth/register`, `/api/auth/refresh` — 5 percobaan/menit per IP + **penundaan konstan 300 ms pada setiap balasan gagal** (meredam brute force jauh lebih efektif daripada counter di lingkungan serverless) + `log auth.rate_limit`. - **Saat naik Vercel Pro (P0, 1 jam):** tambahkan Opsi **A** sebagai lapisan luar: 20 req/menit/IP ke `/api/auth/*`, 100 req/menit/IP global, challenge untuk ASN datacenter. **Dokumentasikan rule-nya di dokumen ini** karena tidak tersimpan di git. - **T2 (P2, 1 hari):** Opsi **C** untuk kuota per-user pada endpoint tulis (`/api/hasil-main`, `/api/pesanan`).

Trade-off khusus `/api/auth/*`: endpoint ini **tidak** lewat `proxy.ts` (matcher mengecualikan `api`), jadi rate limit **harus** dipasang di route handler-nya sendiri atau di lapisan edge Firewall — bukan di middleware. Ini sering salah diasumsikan; catat eksplisit di dokumen.

B.6 Pola "jangan telan error" pengganti `catch {}` kosong

Tiga aturan, satu tabel outbox, satu helper.

Aturan 1 — kegagalan diam dilarang. Semua `const { data } = await supabase...` menjadi `const { data, error } = ...` dan `error` **harus** ditangani salah satu dari: (a) `throw`, (b) dilaporkan lewat `log('error', ...)` + Sentry, (c) `bolehGagal()` eksplisit dengan komentar alasan.

Aturan 2 — jalur keuangan tidak boleh punya jalan diam sama sekali. Kegagalan `transaksi_keuangan` masuk **outbox** yang bisa di-replay.

Aturan 3 — kegagalan boleh tidak memblok user, tapi TIDAK BOLEH tidak tercatat.

Artefak: tabel outbox (bagian dari migrasi `0087`)

```

-- 0087_ledger_gagal.sql – outbox kegagalan tulis keuangan. Tanpa ini, catch{} kosong = uang hilang.
create table if not exists public.ledger_gagal (
  id          uuid primary key default gen_random_uuid(),
  payload     jsonb          not null,
  pesan       text           not null,
  kode        text,
  dibuat_at   timestamptz not null default now(),
  diselesaikan_at timestamptz,
  catatan     text
);
create index if not exists idx_ledger_gagal_belum
  on public.ledger_gagal(dibuat_at desc) where diselesaikan_at is null;

alter table public.ledger_gagal enable row level security;
drop policy if exists "admin kelola ledger_gagal" on public.ledger_gagal;
create policy "admin kelola ledger_gagal" on public.ledger_gagal
  for all to authenticated using (public.is_admin()) with check (public.is_admin());

insert into public.schema_migrations (versi, nama) values ('0087','ledger_gagal')
  on conflict (versi) do nothing;

```

Artefak: `src/lib/data/ledger.ts` versi baru

```

// src/lib/data/ledger.ts – kegagalan keuangan TIDAK BOLEH silent.
// Kontrak baru: fungsi mengembalikan { ok }. Pemanggil boleh memilih tidak menggagalkan
// transaksi inti, tapi kegagalan SELALU (1) masuk outbox, (2) ter-log, (3) memicu alert P0.
import * as Sentry from '@sentry/nextjs';
import { log } from '@lib/obs/log';
import { createClient } from '@lib/supabase/server';
import { tanggalWIB } from '@lib/domain/gamifikasi';

type Supa = Awaited<ReturnType<typeof createClient>>;
type Hasil = { ok: true } | { ok: false; pesan: string };

export async function catatLedger(s: Supa, row: BarisLedger): Promise<Hasil> {
  const baris = {
    arah: row.arah,
    kategori: row.kategori,
    jumlah: Math.max(0, Math.floor(Number(row.jumlah) || 0)),
    tanggal: row.tanggal ?? tanggalWIB(),
    metode: row.metode ?? null,
    keterangan: row.keterangan ?? null,
    ref_tipe: row.ref_tipe ?? null,
    ref_id: row.ref_id ?? null,
    lampiran_url: row.lampiran_url ?? null,
    pic: row.pic ?? null,
    dibuat_oleh: row.dibuat_oleh ?? null,
  };

  const { error } = await s.from('transaksi_keuangan').insert(baris);
  if (!error) return { ok: true };

  // (1) OUTBOX – supaya baris keuangan bisa di-replay admin, bukan hilang
  const { error: errOutbox } = await s
    .from('ledger_gagal')
    .insert({ payload: baris, pesan: error.message, kode: error.code ?? null });

  // (2) LOG TERSTRUKTUR – inilah pengganti `catch {}` kosong
  log('error', 'ledger.gagal', {
    arah: baris.arah, kategori: baris.kategori, jumlah: baris.jumlah,
    ref_tipe: baris.ref_tipe, ref_id: baris.ref_id,
    kode: error.code, pesan: error.message,
    outbox: errOutbox ? 'GAGAL_JUGA' : 'tersimpan',
  });

  // (3) ALERT P0
  Sentry.captureException(new Error(`ledger.gagal: ${error.message}`), {
    level: 'fatal',
    tags: { area: 'keuangan', outbox: errOutbox ? 'gagal' : 'ok' },
    extra: { ref_tipe: baris.ref_tipe, ref_id: baris.ref_id, jumlah: baris.jumlah },
  });

  // (4) Tidak throw: transaksi inti user tetap selesai. Tapi pemanggil TAHU dan bisa
  // menampilkan "catatan keuangan tertunda" di panel admin.
  return { ok: false, pesan: error.message };
}

```

Catatan penting untuk dokumen: komentar asli di `ledger.ts` ("*agar transaksi inti tak rusak bila migrasi 0052 belum dijalankan*") adalah **gejala dari masalah Bab C**, bukan alasan yang sah. Setelah pelacak migrasi + urutan rilis expand→migrate→contract berjalan, alasan itu hilang dan `try/catch` defensif ini boleh dipersempit.

Artefak: helper umum `src/lib/obs/wajib.ts`


```
// src/lib/obs/wajib.ts – dua pintu keluar yang sah untuk error Supabase. Tidak ada pintu ketiga.
import * as Sentry from '@sentry/nextjs';
import { log } from '@lib/obs/log';

type Res<T> = { data: T | null; error: { message: string; code?: string } | null };

/** Data WAJIB ada. Gagal = throw (ditangkap error.tsx + onRequestError). */
export async function harus<T>(p: PromiseLike<Res<T>>, konteks: string): Promise<T> {
  const { data, error } = await p;
  if (error) { log('error', 'db.error', { konteks, kode: error.code, pesan: error.message }); throw
new Error(`${konteks}: ${error.message}`); }
  if (data === null) throw new Error(`${konteks}: data kosong`);
  return data;
}

/** Data OPSIONAL. Gagal = TIDAK throw, TAPI tetap dilaporkan. Wajib sertakan `alasan`. */
export async function bolehGagal<T>(p: PromiseLike<Res<T>>, konteks: string, alasan: string):
Promise<T | null> {
  const { data, error } = await p;
  if (error) {
    log('warn', 'db.gagal_lunak', { konteks, alasan, kode: error.code, pesan: error.message });
    Sentry.captureMessage(`db.gagal_lunak: ${konteks}`, { level: 'warning', extra: { alasan, pesan:
error.message } });
    return null;
  }
  // Deteksi pemotongan silent PostgREST 1000 baris
  if (Array.isArray(data) && data.length === 1000) log('warn', 'postgrest.terpotong', { konteks });
  return data;
}
```

Dampak: mengubah kelas kegagalan terbesar (silent) menjadi terlihat, tanpa refactor besar — cukup ganti pemanggilan di jalur kritis (`ledger.ts` , `skor-core.ts` , 19 route API, halaman keuangan). **Effort:** helper 2 jam + penerapan jalur kritis 1 hari. **Prasyarat:** Sentry. **P0** untuk `ledger.ts` + 19 route API; **P1** untuk sisanya.

4. Bab C — Backup, DR & disiplin rilis/migrasi

C.1 Strategi backup

C.1.a Apa yang tersedia

Kapabilitas	Supabase Free	Supabase Pro 
Backup harian otomatis	tidak ada	ada, retensi 7 hari
Unduh backup	tidak	ya (dashboard)
PITR	tidak	add-on ~\$100/bln (jendela 7 hari)
Retensi log	1 hari	7 hari
<code>pg_dump</code> manual	ya (jalur satu-satunya)	ya
Backup file Storage	tidak dicakup backup DB	tidak dicakup backup DB

Dua kesimpulan yang harus tertulis tebal di dokumen: 1. **Kondisi sekarang = nol backup.** Satu perintah salah di SQL Editor (aktivitas rutin: 86 migrasi manual) = kehilangan permanen semua data ortu, anak, dan

keuangan. Naik Supabase Pro adalah **\$25/bln untuk menghilangkan risiko kebangkrutan produk** — item termurah dengan dampak tertinggi di seluruh dokumen ini. 2. **Backup DB Supabase TIDAK mencakup file di bucket `aset`**. Bukti bayar ortu dan nota keuangan **tidak** ter-backup oleh apa pun sekarang. Butuh jalur backup Storage terpisah.

C.1.b Artefak: `scripts/backup-db.sh` (dijalankan owner)

```
#!/usr/bin/env bash
# scripts/backup-db.sh — backup mandiri KidzPlayful. Jalankan dari laptop owner tiap Jumat (~10
# menit).
#
# Prasyarat:
# 1) postgresql-client >= versi server (pg_dump versi lama akan menolak).
# 2) 7-Zip di PATH (enkripsi).
# 3) File rahasia DI LUAR repo: ~/.kidzplayful-backup.env (chmod 600)
# PGURI="postgresql://postgres.<ref>:<pass>@aws-0-ap-south-1.pooler.supabase.com:5432/postgres"
# PASSPHRASE="..." # simpan di password manager, JANGAN di repo
# CATATAN: pakai port 5432 (session pooler / direct). Port 6543 (transaction pooler)
# TIDAK mendukung pg_dump. Koneksi direct kini IPv6-only — bila jaringan owner
# IPv4-only, gunakan hostname pooler seperti di atas, atau add-on IPv4 (~$4/bln 🧐).
set -euo pipefail
source ~/.kidzplayful-backup.env

STAMP=$(date +%Y%m%d-%H%M)
DIR="/d/backup-kidzplayful/$STAMP"
mkdir -p "$DIR"

echo "[1/5] schema public (struktur + data)"
pg_dump "$PGURI" --format=custom --no-owner --no-privileges \
  --schema=public --file="$DIR/public.dump"

echo "[2/5] schema auth (akun ortu — TANPA INI restore = semua user hilang)"
pg_dump "$PGURI" --format=custom --no-owner --no-privileges \
  --schema=auth --file="$DIR/auth.dump"

echo "[3/5] schema storage (metadata objek; file fisik lihat backup-storage)"
pg_dump "$PGURI" --format=custom --no-owner --no-privileges \
  --schema=storage --file="$DIR/storage-meta.dump"

echo "[4/5] manifest verifikasi (dipakai saat uji restore)"
psql "$PGURI" -Atc "
  select 'profiles='||(select count(*) from profiles)
  || ' anak='||(select count(*) from anak)
  || ' hasil_main='||(select count(*) from hasil_main)
  || ' transaksi_keuangan='||(select count(*) from transaksi_keuangan)
  || ' pendaftaran_event='||(select count(*) from pendaftaran_event)
  || ' pesanan='||(select count(*) from pesanan)
  || ' total_masuk='||(select coalesce(sum(jumlah),0) from transaksi_keuangan where arah='masuk')
  || ' total_keluar='||(select coalesce(sum(jumlah),0) from transaksi_keuangan where arah='keluar')
  || ' migrasi='||(select coalesce(max(versi),'-') from schema_migrations)
" > "$DIR/MANIFEST.txt"
cat "$DIR/MANIFEST.txt"

echo "[5/5] enkripsi (isi mengandung data pribadi anak & bukti transfer)"
7z a -t7z -mhe=on -p"$PASSPHRASE" "$DIR.7z" "$DIR" >/dev/null
rm -rf "$DIR"
echo "SELESAI: $DIR.7z ($(du -h "$DIR.7z" | cut -f1))"
```

C.1.c Artefak: backup Storage (bucket `aset`)

```
# scripts/backup-storage.sh - bucket `aset` TIDAK tercakup backup DB Supabase.
# Supabase Storage mendukung protokol S3 → pakai rclone (inkremental, murah).
#   rclone config: tipe s3, provider Other,
#   endpoint   = https://<ref>.supabase.co/storage/v1/s3
#   region     = ap-south-1
#   access_key/secret = S3 Access Keys dari Supabase Dashboard → Storage → S3
set -e || pipefail
rclone sync "supabase:aset" "/d/backup-kidzplayful/aset" \
  --progress --transfers 8 --checksum \
  --exclude "uji/**" # jangan backup sampah skrip tools/
du -sh /d/backup-kidzplayful/aset
```

C.1.d Di mana disimpan (3-2-1 versi hemat untuk 1 orang)

Salinan	Lokasi	Frekuensi	Catatan
1	D:\backup-kidzplayful\ (laptop owner)	mingguan	sumber utama, terenkripsi 7z AES-256
2	Google Drive owner, folder kidzplayful-backup	mingguan, upload manual/rclone	hanya file .7z terenkripsi
3	Disk eksternal / flashdisk	bulanan	offline, aman dari ransomware & salah hapus akun

Larangan tegas (tuliskan di dokumen): `.7z` backup **TIDAK BOLEH** masuk repo (publik!). Tambahkan ke `.gitignore`: `*.dump`, `*.7z`, `backup/`, `.kidzplayful-backup.env`. Passphrase hanya di password manager.

Retensi: mingguan 8 minggu, bulanan 12 bulan, tahunan permanen (data keuangan → kebutuhan pembukuan).

C.1.e Jadwal uji restore — backup tanpa uji restore = tidak ada backup

Kapan	Ruang lingkup	Target	Durasi	Prio
Sekali sekarang	restore penuh ke Postgres lokal (Docker)	buktikan dump bisa dibaca	2 jam	P0
Bulanan, Sabtu pertama	restore penuh ke proyek Supabase kedua (<code>kidzplayful-restore-test</code> , Free)	ukur RTO nyata	1,5 jam	P1
Kuartalan	restore + jalankan app lokal terhubung ke DB hasil restore + smoke 5 alur	buktikan aplikasi hidup, bukan hanya data	3 jam	P1
Setiap kali skema berubah besar	restore dump terbaru + jalankan migrasi berikutnya	buktikan urutan rilis	1 jam	P2

Checklist verifikasi uji restore (WAJIB dicentang, catat waktunya): 1. `pg_restore` selesai tanpa error fatal. 2. Bandingkan output `MANIFEST.txt` dengan hasil query yang sama di DB restore → **8 angka harus identik**, termasuk `total_masuk` & `total_keluar` `transaksi_keuangan`. 3. `select max(versi) from schema_migrations` sama dengan manifest. 4. Login 1 akun ortu uji berhasil (bukti `auth.users` + `profiles` ikut ter-restore). 5. Buka `/pilih-anak`, `/main/<anakId>`, `/admin/keuangan` → tidak ada halaman error. 6. Buka 1 URL `bukti/` dari backup Storage → file ada. 7. **Catat total menit dari mulai sampai poin 6 selesai. Angka itu = RTO nyata Anda.** Tulis di tabel riwayat uji restore di dokumen ini.

C.2 RPO / RTO realistis per tier

Tier	Backup aktif	RPO (kehilangan data maks.)	RTO (waktu pulih)	Yang membuat angka ini realistis
T0 sekarang (Free, tanpa apa pun)	—	∞ (total)	∞	tidak ada yang bisa dipulihkan
T0 target (Pro + dump mingguan)	daily 7 hari + dump mingguan	24 jam	4–8 jam	1 orang, runbook belum terlatih, restore manual dari dashboard
T1	idem + runbook tertulis + uji restore bulanan	24 jam	2–4 jam	RTO turun karena runbook sudah dilatih, bukan karena teknologi
T2	+ PITR (~\$100/bln)	≤ 1 jam	≤ 1 jam	PITR + staging identik untuk verifikasi cepat
T3	+ read replica + staging identik + smoke otomatis	≤ 5 menit	≤ 30 menit	promosi replica + DNS/env switch

Poin yang harus tegas: RTO turun bukan karena membeli fitur, tapi karena **runbook yang sudah pernah dijalankan**. Owner tunggal tanpa latihan restore = RTO realistis 8 jam walaupun ada PITR.

Artefak: Runbook DR (ringkas, tempel di dokumen)

RUNBOOK: DB PRODUKSI RUSAK / DATA TERHAPUS

Prasyarat: akses dashboard Supabase, passphrase backup, laptop dengan psql.

0. (2 menit) BEKUKAN KERUSAKAN

- Vercel → Project → Settings → Deployment Protection: aktifkan, atau set env NEXT_PUBLIC_MODE_PEMELIHARAAN=1 lalu redeploy (mencegah tulis baru).
- Catat waktu insiden (UTC & WIB). Ini menentukan titik PITR.

1. (5 menit) TENTUKAN RUANG LINGKUP

- 1 tabel / beberapa baris → Jalur A (restore selektif)
- seluruh DB → Jalur B (restore penuh)
- file storage hilang → Jalur C

2A. JALUR A – selektif (30-60 menit)

- Restore dump terakhir ke Postgres lokal (Docker).
- Ekspor HANYA tabel/baris terdampak: `pg_dump --table=<t> --data-only`
- Import ke produksi di dalam transaksi; verifikasi count sebelum COMMIT.

2B. JALUR B – penuh (2-6 jam)

- Pro tanpa PITR : Dashboard → Database → Backups → pilih tanggal → Restore.
- Pro + PITR : Restore to point in time → 5 menit SEBELUM waktu insiden.
- Tanpa keduanya : buat proyek baru, `pg_restore public+auth+storage-meta`, update env Vercel (URL/anon/service key), redeploy.
- Jalankan checklist verifikasi C.1.e poin 2-6.

2C. JALUR C – storage (30 menit)

- `rclone copy /d/backup-kidzplayful/aset supabase:aset --immutable`

3. (10 menit) BUKA KEMBALI

- Nonaktifkan mode pemeliharaan. Pantau `/api/health/db` + Sentry 30 menit.

4. (esok hari) POST-MORTEM

- Tulis di dokumen ini: penyebab, RPO nyata, RTO nyata, 1 perubahan pencegah.

C.3 Pelacak migrasi + pola idempoten

C.3.a Masalah yang harus diselesaikan (fakta)

86 file dijalankan manual, tanpa pelacak versi. 13 `create table` tanpa `if not exists` (7 file), 8 `create index` tanpa `if not exists`, ~160 `create policy` sementara hanya 31 file memakai `drop policy if exists`. Konsekuensi presisi: **rerun 0001→0086 pada DB terisi sebagian pasti gagal di tengah dan berhenti pada state parsial** — artinya *hari ini tidak ada cara terpercaya untuk membangun ulang produksi dari repo*. Itu berarti: **backup DB Anda saat ini lebih bernilai daripada repo Anda**, kondisi yang harus dibalik.

C.3.b Artefak: tabel pelacak

```
-- supabase/migrations/0000_baseline.sql
-- Dijalankan SEKALI di produksi. Fungsi: (1) buat pelacak versi,
-- (2) catat retroaktif 0001-0086 sebagai "sudah jalan" TANPA menjalankannya ulang.
-- Ini pendekatan termurah: nol file lama yang perlu ditulis ulang.
create table if not exists public.schema_migrations (
  versi          text primary key,          -- '0001' ... '0086' ... '0087'
  nama           text          not null,
  checksum       text,                  -- sha256 isi file; mendeteksi file diubah setelah jalan
  dijalankan_at  timestamptz not null default now(),
  dijalankan_oleh text          not null default current_user,
  durasi_ms      integer,
  retroaktif     boolean         not null default false -- true = dicatat, bukan dieksekusi
);

comment on table public.schema_migrations is
  'Pelacak migrasi. Baris retroaktif=true dicatat pada 0000_baseline tanpa dieksekusi.';

-- RLS aktif TANPA policy: hanya service_role / SQL Editor (postgres) yang bisa akses.
alter table public.schema_migrations enable row level security;

-- Catat 0001..0086 secara retroaktif.
insert into public.schema_migrations (versi, nama, retroaktif)
select lpad(g::text, 4, '0'), 'baseline-retroaktif', true
from generate_series(1, 86) g
on conflict (versi) do nothing;
```

Kenapa `generate_series` dan bukan daftar 86 nama? Karena tujuan baseline hanya **menghentikan runner menjalankan ulang** file lama. Nama asli tetap ada di git. Kalau ingin nama yang benar, runner (`tools/migrate.mjs --baseline`) bisa membaca nama file dan `update` kolom `nama` + `checksum` sekali jalan (tambahan 15 menit).

C.3.c Artefak: template file migrasi baru (wajib mulai `0087`)

```
-- supabase/migrations/0087_<nama>.sql
-- ATURAN WAJIB (berlaku sejak 0087):
-- 1) Semua DDL idempoten: create table/index/type IF NOT EXISTS.
-- 2) Setiap create policy DIDAHULUI drop policy if exists dengan nama identik.
-- 3) Kolom baru: nullable ATAU punya default. TIDAK BOLEH langsung NOT NULL.
-- 4) Berkas ini harus AMAN dijalankan dua kali berturut-turut.
-- 5) Baris terakhir mencatat diri ke schema_migrations.
-- 6) Bila memakai CREATE INDEX CONCURRENTLY, tambahkan penanda di baris 1:
--     -- no-transaction
--     (runner akan menjalankan file ini di luar transaksi)

begin;

alter table public.event add column if not exists kuota_pendamping integer;

create index if not exists idx_event_kuota on public.event(kuota_pendamping)
  where kuota_pendamping is not null;

drop policy if exists "baca event kuota" on public.event;
create policy "baca event kuota" on public.event
  for select to authenticated using (true);

insert into public.schema_migrations (versi, nama) values ('0087','event_kuota_pendamping')
  on conflict (versi) do nothing;

commit;
```

C.3.d Artefak: runner `tools/migrate.mjs` (garis besar, ~90 baris)

```
// tools/migrate.mjs – jalankan hanya migrasi yang belum tercatat, SATU TRANSAKSI PER FILE.
// Pakai: node tools/migrate.mjs --db-url "$PGURI" [--dry-run] [--sampai 0090]
// Konsekuensi: state parsial tidak mungkin lagi (DDL Postgres transaksional).
// - baca supabase/migrations/*.sql,urut nama file
// - SELECT versi FROM schema_migrations -> set "sudah"
// - untuk setiap file belum ada:
//   * hitung sha256
//   * jika baris 1 mengandung '-- no-transaction' -> eksekusi apa adanya
//   selain itu -> BEGIN; <isi>; INSERT schema_migrations(versi,nama,checksum,durasi_ms); COMMIT;
//   * gagal -> ROLLBACK, cetak versi + pesan, EXIT 1 (jangan lanjut file berikutnya)
// - untuk setiap file yang SUDAH tercatat tapi checksum beda -> WARNING keras
// ("file migrasi yang sudah jalan diubah – buat file baru, jangan edit yang lama")
```

Verifikasi idempotensi di CI (ini yang mencegah kambuh) — job baru di [.github/workflows/ci.yml](https://github.com/supabase/workflows/ci.yml) :

```

migrasi:
  runs-on: ubuntu-latest
  services:
    postgres:
      image: postgres:16
      env: { POSTGRES_PASSWORD: postgres }
      ports: ['5432:5432']
      options: >-
        --health-cmd pg_isready --health-interval 5s --health-timeout 5s --health-retries 10
  steps:
    - uses: actions/checkout@v5
    - uses: actions/setup-node@v5
      with: { node-version: 20, cache: npm }
    - run: npm ci
    # Prasyarat: stub minimal utk objek Supabase (auth.users, storage.*, auth.uid()).
    - name: Siapkan stub Supabase
      run: psql "$DB" -f supabase/ci/stub-supabase.sql
      env: { DB: 'postgresql://postgres:postgres@localhost:5432/postgres' }
    - name: Migrasi dari NOL (harus sukses)
      run: node tools/migrate.mjs --db-url "$DB"
      env: { DB: 'postgresql://postgres:postgres@localhost:5432/postgres' }
    - name: Migrasi DIULANG (harus no-op, bukan error) # <-- gerbang idempotensi
      run: node tools/migrate.mjs --db-url "$DB"
      env: { DB: 'postgresql://postgres:postgres@localhost:5432/postgres' }

```

Pendekatan baseline	Effort	Plus	Minus	Putusan
A. <code>0000_baseline.sql</code> + tabel sendiri + <code>tools/migrate.mjs</code>	6–8 jam	tidak me-rename/menulis ulang satu pun dari 86 file; runner dikuasai penuh; checksum drift; kompatibel dengan nama <code>0001_init.sql</code>	menulis runner sendiri (~90 baris)	PILIH INI
B. Supabase CLI <code>db push</code> + <code>migration repair --status applied</code>	4 jam + rename	runner gratis & teruji, <code>supabase_migrations.schema_migrations</code> bawaan	CLI menuntut nama <code><timestamp>_nama.sql</code> → harus me-rename 86 file (riwayat git berisik, semua referensi dokumen ke "migrasi 0044/0052" jadi salah)	cadangan
C. Tulis ulang 86 file jadi idempoten	2–3 hari	repo bisa merekonstruksi DB sepenuhnya	mahal, berisiko tinggi, nilai rendah (produksi sudah ada)	tolak

Prasyarat A: `pg` (npm) atau `psql` di mesin owner; `PGURI` di file rahasia luar repo. **Prioritas P0.**

C.4 Urutan rilis: hentikan kode mendahului migrasi

Akar masalah (fakta)

Vercel auto-deploy tiap push `master`, sedangkan migrasi dijalankan manual **setelahnya** → **jendela race struktural**. Insiden nyata `kuota_*` adalah wujudnya. Mitigasi saat ini (konvensi "kolom baru harus toleran" + `try/catch`) menyembunyikan gejala, tidak menutup jendela — dan `ledger.ts` **adalah bukti biaya sampingannya**: kegagalan keuangan ditelan karena takut migrasi `0052` belum jalan.

Pola wajib: `expand` → `migrate` → `contract`

Fase	Tindakan	Yang dilakukan	Aturan
1. EXPAND	migrasi saja , dijalankan DULU di produksi	tambah kolom nullable/berdefault, tabel baru, index baru	Tidak ada perubahan kode di rilis ini. Selalu backward-compatible.
2. DEPLOY	kode yang menulis dan membaca kolom baru	dipush setelah fase 1 terbukti (<code>/api/health/db</code> = 200, <code>MIGRASI_MINIMAL</code> naik)	kode tetap toleran terhadap <code>null</code> (bukan terhadap <i>kolom tidak ada</i>)
3. BACKFILL	isi data lama	batch (<code>limit</code>), via runner atau cron, idempoten	jangan <code>update</code> satu tabel besar dalam satu perintah
4. CONTRACT	migrasi pengetat	<code>set not null</code> , <code>drop column</code> lama, hapus lapisan toleransi	minimal 1 rilis setelah fase 3 , dan hanya bila fase 2–3 sudah stabil $\geq$ 1 minggu

Mekanik penegakan, dari termurah:

#	Langkah	Effort	Dampak	Prio
1	Aktifkan branch protection <code>master</code> (PR wajib, CI wajib hijau, linear history) — sekarang masih "saran" di docs	15 menit	tidak ada lagi push langsung ke produksi	P0
2	Template PR dengan checklist: " <i>Migrasi mana yang dibutuhkan? Sudah dijalankan di produksi? Tempel output <code>tools/migrate.mjs</code> ."</i>	30 menit	memaksa urutan jadi eksplisit	P0
3	Env <code>MIGRASI_MINIMAL</code> di Vercel + cek di <code>/api/health/db</code> (Bab B)	sudah termasuk	deteksi race < 5 menit dengan alert P0	P0
4	Uji di proyek Supabase beta dulu (docs environment beta sudah ada)	1 hari	migrasi tidak pernah pertama kali menyentuh produksi	P1
5	Matikan Vercel Git auto-deploy production ; deploy lewat GitHub Actions: <code>tsc → test → build → migrate → vercel deploy --prod</code>	3–4 jam	pencegahan , bukan deteksi: mustahil kode mendahului migrasi	P1

Kapan "kolom toleran" TETAP dipakai (jangan dibuang, ubah perannya): - **Ya** — selama jendela fase 2 (kolom sudah ada tapi masih `null` untuk baris lama); untuk field opsional/kosmetik; untuk kompatibilitas app mobile Flutter yang versinya tertinggal di HP pengguna (**alasan permanen dan sah**, karena rilis app store tidak bisa dipaksa). - **Tidak** — sebagai satu-satunya pengaman race; di jalur keuangan (`ledger.ts` , `transaksi_keuangan` , `pesanan`) di mana kegagalan **harus** berbunyi; sebagai alasan `catch {}` kosong.

C.5 Membereskan `tools/` — ini insiden keamanan aktif

C.5.a Klasifikasi

- **Tingkat 1 — kebocoran kredensial (P0, jam ini):** 56 kemunculan `admin@kidzplayful.app` + password di **23 skrip** di **repo publik**. Terkonfirmasi di `tools/admin_check.mjs` dkk. Asumsikan **sudah bocor** (GitHub code search mengindeks repo publik dalam hitungan menit).
- **Tingkat 2 — skrip menulis ke produksi (P0/P1):** 10 skrip menarget URL produksi; **11 skrip menunjuk localhost** **TAPI** memakai Supabase **PRODUKSI** — kategori paling berbahaya karena *tampak* aman.
- **Tingkat 3 — sampah data uji (P1):** produk uji, pendaftaran uji, file bukti terunggah ke bucket produksi.

C.5.b Langkah, berurutan

#	Langkah	Detail	Effort	Prasyarat	Prio
1	Rotasi password admin	Supabase → Auth → Users → <code>admin@kidzplayful.app</code> → set password baru (generator, ≥ 24 karakter) → simpan hanya di password manager	15 menit	—	P0
2	Audit jejak penyalahgunaan	<code>auth.users.last_sign_in_at</code> akun admin; <code>select * from aktivitas order by dibuat_at desc limit 200</code> ; baris <code>transaksi_keuangan / pesanan / postingan</code> tak dikenal; <code>select name, created_at from storage.objects order by created_at desc limit 100</code> (file asing); dan <code>git log -p -S 'service_role'</code> — anon key aman dipublikasikan, service role TIDAK	1 jam	—	P0
3	Ganti identitas admin	buat akun admin baru dengan email tak mudah diduga (bukan pola <code>admin@<domain></code>); turunkan akun lama jadi non-admin (jangan hapus — ada FK <code>dibuat_oleh</code>)	30 menit	1	P0
4	Kredensial → env var	23 skrip membaca <code>process.env.KP_EMAIL</code> , <code>KP_PASSWORD</code> , <code>KP_BASE_URL</code> , <code>KP_SUPABASE_URL</code> ; fail-fast bila kosong; nilai di <code>.env.tools.local</code> (gitignored). Buat <code>tools/_env.mjs</code> bersama agar tidak menyalin 23×	2–3 jam	1	P0
5	Guard anti-produksi	di <code>tools/_env.mjs</code> : bila <code>KP_BASE_URL / KP_SUPABASE_URL</code> mengandung domain produksi → <code>process.exit(1)</code> kecuali <code>KP_ALLOW_PROD=1</code> diset eksplisit. Ini yang menutup 11 skrip "localhost tapi Supabase produksi" .	1 jam	4	P0
6	Akun uji khusus di beta	<code>qa@kidzplayful.test</code> hanya di proyek Supabase beta; skrip default menunjuk beta	3–4 jam	environment beta	P1
7	Bersihkan data uji produksi	skrip inventarisasi (nama berprefiks <code>[UJI]</code>), akun uji, objek <code>storage</code> dari skrip) → tinjau manual → hapus idempoten. Konvensi baru: semua data buatan skrip berprefiks <code>[UJI] / uji/</code> supaya bisa dihapus dengan aman	2–3 jam	6	P1
8	Gitleaks di CI + pre-commit	job <code>gitleaks</code> di <code>ci.yml</code> (gagalkan PR bila ada secret) + hook lokal	1 jam	—	P1
9	Dependabot + npm audit di CI	belum ada sama sekali	30 menit	—	P1
10	Riwayat git	password tetap ada di riwayat walau file diubah. <code>git filter-repo</code> + force-push mungkin (repo publik, tanpa kolaborator) tapi fork & cache GitHub tetap menyimpan → rotasi (langkah 1) adalah satu-satunya mitigasi nyata ; rewrite riwayat nilainya rendah	2 jam	1	P2 (opsional)
11	Aturan repo publik	tulis di <code>CLAUDE.md</code> : tidak ada kredensial apa pun di repo; semua <code>.env*</code> di <code>.gitignore</code> ; service role hanya di Vercel env; anon key boleh publik karena dilindungi RLS	30 menit	—	P1

C.5.c Perbaikan Storage yang menyertai (ikut di bab ini karena satu insiden data)

#	Perbaikan	Effort	Prio
1	<code>file_size_limit</code> + <code>allowed_mime_types</code> pada bucket + validasi ukuran di klien — detail & prasyaratnya di F.1/F.2 (jangan jalankan SQL-nya sebelum <code>src/lib/upload.ts</code> ter-deploy)	1 jam	P1
2	Pisahkan bucket <code>privat</code> (<code>public=false</code>) untuk <code>bukti/</code> & <code>nota/</code> — rancangan lengkap 3 fase + policy + <code>urlLampiran()</code> ada di F.3. Catatan: jangan kirim signed URL ke <code>next/image</code> .	1–1,5 hari	P0 (≤ 2 minggu)
3	Hapus berkas lama saat mengganti, atau path ber-entitas + <code>upsert:true</code> — peringatan penting di F.4 : aset game dirujuk dari dalam jsonb <code>paket_aset.butir</code> , jadi pembersih naif akan menghapus semuanya	2 jam	P1
4	Sertakan bucket ke jalur backup (<code>scripts/backup-storage.sh</code>)	1 jam	P0

5. Bab D — Lapisan data: index, RLS, agregasi

Tiga pekerjaan di bab ini punya sifat berbeda dan **jangan disamakan prioritasnya**: index adalah kemenangan murah (menit), RLS adalah pekerjaan berisiko yang butuh uji per peran (setengah hari), dan agregasi SQL adalah **koreksi kebenaran data** — bukan optimasi.

D.1 Index: satu migrasi, semua yang benar-benar dibutuhkan

Repo punya ~45 index, tapi satu-satunya migrasi khusus performa (`0039_perf_index.sql`) berhenti di era sebelum store & keuangan ada. Sejak itu tabel `event`, `produk`, `tema`, dan `aset` tidak pernah mendapat index sama sekali.

Yang membuat ini lebih parah dari kelihatannya: kolom `status` pada `event`, `produk`, `kelas_bermain`, `tema`, `paket_aset`, dan `video` dipakai **dua kali** per query — sekali dari `.eq('status', ...)` di kode, dan sekali lagi di dalam ekspresi `USING` policy RLS. Tanpa index, dua-duanya sequential scan.

Artefak: `supabase/migrations/0087_perf_index_2.sql`

```
-- 0087_perf_index_2.sql – index untuk kolom yang sering difilter/diurutkan tapi belum ter-index.
-- Idempoten: aman dijalankan berulang. Semua CREATE INDEX IF NOT EXISTS.
--
-- CATATAN OPERASIONAL: pada tabel besar, CREATE INDEX biasa mengambil lock tulis.
-- Tabel di bawah masih kecil (< puluhan ribu baris) sehingga aman dijalankan langsung.
-- Bila suatu tabel sudah besar, jalankan baris itu sendirian dengan CONCURRENTLY
-- DI LUAR transaksi (lihat template C.3.c penanda '-- no-transaction').
```

```
begin;
```

```
-- — event: TIDAK punya index apa pun selain PK —————
-- Melayani: event.ts:10, publik.ts:17, komunitas.ts:41, admin-event.ts:13, guru.ts:20
-- (.eq('status','tampil') + .order('tanggal'))
create index if not exists idx_event_status_tanggal
  on public.event(status, tanggal desc);
-- Melayani: policy RLS "event baca peserta" (0068:5-9) yang meng-exists ke pendaftaran_event
-- – composite ini yang sebelumnya tidak ada, hanya dua index terpisah.
create index if not exists idx_pendaftaran_event_event_ortu
  on public.pendaftaran_event(event_id, ortu_id);

-- — pendaftaran_event: index lama (event_id,status) salah leading column —
-- Melayani: admin-event.ts:43 (.eq('status','menunggu') TANPA event_id) – badge notifikasi.
-- Partial karena 'menunggu' adalah minoritas baris dan satu-satunya nilai yang di-query global.
create index if not exists idx_pendaftaran_event_menunggu
  on public.pendaftaran_event(created_at desc) where status = 'menunggu';
-- Melayani: admin-reminder.ts:20 (.eq('status','diterima') global)
create index if not exists idx_pendaftaran_event_diterima
  on public.pendaftaran_event(event_id) where status = 'diterima';

-- — produk: TIDAK punya index apa pun selain PK —————
-- Melayani: store.ts:9, publik.ts:25, admin-store.ts:10, rekomendasi-item.ts:12
create index if not exists idx_produk_status_created
  on public.produk(status, created_at desc);

-- — tema: TIDAK punya index apa pun selain PK —————
-- Melayani: pustaka.ts:15, tema.ts:8, publik.ts:49, panduan.ts:8, admin/page.tsx:12
create index if not exists idx_tema_status
  on public.tema(status);
-- Melayani: tema "minggu ini" – partial, karena hanya sedikit baris bernilai true
create index if not exists idx_tema_minggu_ini
  on public.tema(id) where is_minggu_ini = true;

-- — kelas_bermain: hanya ada (created_at desc) —————
-- Melayani: kelas-bermain.ts:7, publik.ts:33,41, komunitas.ts:40
create index if not exists idx_kelas_status_created
  on public.kelas_bermain(status, created_at desc);

-- — paket_aset & video —————
-- Melayani: pustaka.ts:24, tema.ts:13, publik.ts:50 (+ policy RLS status)
create index if not exists idx_paket_aset_status
  on public.paket_aset(status);
-- Melayani: video.ts:10-11 (.eq kategori + status + link_ok, .order('urutan'))
create index if not exists idx_video_kategori_urutan
  on public.video(kategori, urutan) where status = 'tampil' and link_ok is not false;

-- — hasil_main: tabel TERCEPAT tumbuh —————
-- Melayani: skor-core.ts:60 & gamifikasi.ts:39 (riwayat satu anak, terurut waktu).
-- Composite (anak_id, tanggal desc) menggantikan hasil_anak_idx(anak_id) yang tak menolong sort.
create index if not exists idx_hasil_main_anak_tanggal
  on public.hasil_main(anak_id, tanggal desc);
-- Melayani: admin/analitik/page.tsx:31 (.gte('tanggal', 30 hari))
create index if not exists idx_hasil_main_tanggal
  on public.hasil_main(tanggal desc);
```

```

-- — pagination & laporan —————
-- Melayani: admin-store.ts:22 (.order + .range), pesanan.ts:11, analitik:33
create index if not exists idx_pesanan_created
  on public.pesanan(created_at desc);
-- Melayani: atribusi.ts:16 (.gte), kpi.ts:164, admin/langganan:73 (.order + .range)
create index if not exists idx_profiles_created
  on public.profiles(created_at desc);
-- Melayani: admin-anak.ts:28, konsultasi.ts:19, pilih-anak:29
create index if not exists idx_anak_created
  on public.anak(created_at desc);
-- Melayani: admin/langganan/page.tsx:79-81 (.lte('aktif_sampai') + order) – deteksi kedaluwarsa
create index if not exists idx_langganan_aktif_sampai
  on public.langganan(aktif_sampai);
-- Melayani: keuangan.ts:231 (.order('created_at'))
create index if not exists idx_aset_created
  on public.aset(created_at desc);

-- — kolom yang dipakai policy RLS tapi belum ter-index —————
-- Melayani: policy 0010:41 (postingan delete milik sendiri) + analitik:34
create index if not exists idx_postingan_ortu
  on public.postingan(ortu_id);
-- Melayani: policy 0010:50 + analitik:35 (.gte created_at)
create index if not exists idx_komentar_created
  on public.komentar(created_at desc);
-- Melayani: policy 0084:35 (voucher_redeem milik sendiri)
create index if not exists idx_voucher_redeem_ortu
  on public.voucher_redeem(ortu_id);
-- Melayani: policy 0067:25 (rekomendasi_item milik sendiri / pemberi)
create index if not exists idx_rekomendasi_item_ortu
  on public.rekomendasi_item(ortu_id);
-- Melayani: boleh_lihat_laporan_anak() (0066:14) – dipakai sebagai predikat RLS di 5 TABEL,
--      jadi index ini menolong lima-limanya sekaligus. Paling berdampak di blok ini.
create index if not exists idx_pendaftaran_konsultasi_anak
  on public.pendaftaran_konsultasi(anak_id);

-- Perencana query butuh statistik baru agar index di atas benar-benar dipakai.
analyze public.event, public.produk, public.tema, public.kelas_bermain,
  public.paket_aset, public.video, public.hasil_main, public.pendaftaran_event,
  public.pesanan, public.profiles, public.anak, public.langganan;

insert into public.schema_migrations (versi, nama) values ('0087','perf_index_2')
  on conflict (versi) do nothing;

commit;

```

Yang sengaja TIDAK di-index

Setiap index memperlambat setiap **insert** / **update** pada tabelnya. Jadi menahan diri juga bagian dari pekerjaan:

Tabel/kolom	Alasan tidak di-index
<code>fokus_area(urutan)</code> , <code>kategori_usia(urutan)</code> , <code>voucher(created_at)</code> , <code>jadwal_psikolog(aktif)</code>	Tabel master, puluhan baris. Sequential scan atas 30 baris lebih cepat daripada lompat index.
<code>aktivitas</code> — jangan tambah	Tabel ini justru kelebihan index: sudah 3 buah, dan semuanya diperbarui pada setiap page view . Rekomendasinya berlawanan arah — turunkan menjadi satu composite (<code>ortu_id, dibuat_at desc</code>), karena (<code>dibuat_at desc</code>) dan (<code>fitur</code>) bisa dilayani index composite itu atau memang jarang dipakai sendirian. Lihat A.3 T1 item 4.
<code>transaksi_keuangan(tanggal)</code>	Sudah ada (<code>0052:26</code>). Masalah tabel ini bukan index, melainkan query tanpa <code>WHERE</code> sama sekali — diselesaikan di D.3 , bukan dengan index.
Kolom jsonb (<code>butir</code> , <code>aktivitas</code> , <code>indikator_perkembangan</code>)	Tidak pernah difilter di dalam DB — hanya diambil lalu diproses di JavaScript. GIN index tidak akan terpakai. Solusinya berhenti mengambilnya (E.2).

Aspek	Isi
Dampak	Menghilangkan sequential scan pada tabel yang dibaca di hampir setiap halaman. Pada data sekarang efeknya masih kecil (tabel kecil); nilainya adalah mencegah tebing — sequential scan biayanya tumbuh linear terhadap jumlah baris, sedangkan index tetap ~konstan.
Effort	15 menit menjalankan + 15 menit verifikasi.
Risiko	Sangat rendah. <code>create index</code> tidak mengubah data. Yang perlu diperhatikan hanya lock tulis sesaat pada tabel besar — catatan sudah ada di kepala berkas.
Verifikasi	<code>explain (analyze, buffers) select ... from event where status='tampil' order by tanggal desc limit 20;</code> → sebelum: <code>Seq Scan on event</code> ; sesudah: <code>Index Scan using idx_event_status_tanggal</code> . Ulangi untuk <code>produk</code> dan <code>hasil_main</code> .
Prioritas	P0 — quick win termurah di seluruh dokumen.

D.2 RLS: angkat evaluasi ke InitPlan

Mekanismenya

Semua fungsi helper (`is_admin()`, `is_guru()`, `is_psikolog()`, `is_superuser()`, `is_investor()`) sudah ditulis dengan benar: `STABLE SECURITY DEFINER`, mencari `profiles` lewat primary key. Itu bagian yang sudah beres.

Masalahnya di **cara memanggilnya**. Saat sebuah fungsi ditulis langsung di ekspresi policy:

```
using (public.is_admin() or ortu_id = auth.uid())          -- dievaluasi PER BARIS
```

PostgreSQL memperlakukannya sebagai bagian dari filter per baris. Bungkus dalam sub-select dan perencanaan mengangkatnya menjadi **InitPlan** — dievaluasi **sekali per query**, hasilnya dipakai ulang:

```
using ((select public.is_admin()) or ortu_id = (select auth.uid()))  -- SEKALI per query
```

Perbedaannya nol pada 10 baris dan besar pada 100.000 baris. Ini perubahan **murni performa** — hasil logisnya identik, karena kedua fungsi memang konstan sepanjang satu query.

Peringatan penting: repo punya ~160 policy. **Jangan mengubah semuanya sekaligus.** Policy adalah kode keamanan; satu salah tulis membuka data keluarga orang lain atau memblokir pengguna yang sah. Kerjakan 8 policy berdampak terbesar di bawah, uji tiap blok dengan empat peran, baru lanjut. Sisanya masuk P2 dengan aturan berhenti: hanya sentuh policy pada tabel yang `n_live_tup` sudah > 1.000.

Delapan policy yang dikerjakan lebih dulu, dan alasannya

#	Tabel	Mengapa paling berdampak
1	hasil_main	Tabel tercepat tumbuh, dan punya 3 policy SELECT permissive yang di-OR — PostgreSQL harus mengevaluasi ketiganya sampai ada yang benar. Salah satunya <code>exists(select 1 from anak ...)</code> per baris, satu lagi <code>boleh_lihat_laporan_anak(anak_id)</code> yang argumennya berubah per baris sehingga <code>STABLE</code> tidak bisa menolong. Dibaca penuh oleh <code>skor-core.ts:60</code> setiap sesi game.
2	anak	Dibaca di hampir setiap halaman lewat <code>getAnakTerjamin</code> .
3	event	Policy <code>exists(... pendaftaran_event ...)</code> per baris, ditambah filter <code>status</code> tanpa index (diperbaiki di D.1).
4	pendaftaran_event	Dibaca penuh oleh 3 reader admin tanpa filter.
5	transaksi_keuangan	Di-full-scan oleh 8 halaman; setiap baris mengevaluasi <code>is_admin()</code> dan <code>is_investor()</code> .
6	pesanan + item_pesanan	<code>item_pesanan</code> punya <code>exists(... pesanan ...)</code> per baris.
7	aktivitas	1 insert per page view + dibaca 3.000–5.000 baris di halaman analitik.
8	profiles	Ditarik seluruh tabel oleh <code>kpi.ts:164</code> dan <code>analitik:36</code> .

Artefak: `supabase/migrations/0088_rls_initplan.sql` (pola untuk tiga tabel pertama)


```

-- 0088_rls_initplan.sql – bungkus fungsi & auth.uid() dalam sub-select agar diangkat ke InitPlan.
-- HASIL LOGIS TIDAK BERUBAH. Ini perubahan performa, bukan perubahan aturan akses.
--
-- CARA KERJA: jalankan BLOK DEMI BLOK, dan setelah tiap blok uji dengan 4 peran
-- (ortu biasa · admin · guru · investor) memakai checklist di bawah tabel ini.
-- JANGAN jalankan seluruh berkas lalu berharap yang terbaik.
--
-- PRASYARAT: 0087 sudah jalan (index pendukung harus ada lebih dulu, kalau tidak
-- policy yang lebih murah hanya memindahkan biaya ke sequential scan).

begin;

-- — BLOK 1 – hasil_main —————
-- Tiga policy SELECT permissive digabung menjadi SATU. Cakupan aksesnya sama persis:
-- (a) ortu pemilik anak (b) admin (c) psikolog yang berhak lihat laporan
-- Dengan satu policy, PostgreSQL berhenti mengevaluasi tiga ekspresi ber-OR per baris.
drop policy if exists "hasil milik ortu" on public.hasil_main;
drop policy if exists "admin baca hasil" on public.hasil_main; -- 0006:8
drop policy if exists "laporan psikolog hasil" on public.hasil_main; -- 0066:27
create policy "hasil_main baca" on public.hasil_main
for select to authenticated
using (
    (select public.is_admin())
    or exists (select 1 from public.anak a
               where a.id = hasil_main.anak_id and a.ortu_id = (select auth.uid()))
    or public.boleh_lihat_laporan_anak(hasil_main.anak_id)
);
-- Urutan OR disengaja: cek termurah lebih dulu (is_admin sekali per query),
-- lalu exists ber-index, dan yang termahal (boleh_lihat_laporan_anak) paling akhir –
-- PostgreSQL melakukan short-circuit, jadi urutan ini menghemat pemanggilan.

drop policy if exists "tuliskan hasil anak sendiri" on public.hasil_main;
create policy "hasil_main tulis" on public.hasil_main
for insert to authenticated
with check (
    exists (select 1 from public.anak a
            where a.id = hasil_main.anak_id and a.ortu_id = (select auth.uid()))
);

-- — BLOK 2 – anak —————
drop policy if exists "anak milik ortu" on public.anak;
create policy "anak milik ortu" on public.anak
for all to authenticated
using (ortu_id = (select auth.uid()) or (select public.is_admin()))
with check (ortu_id = (select auth.uid()) or (select public.is_admin()));

-- — BLOK 3 – event —————
-- Sub-select pada auth.uid(); exists-nya kini dilayani idx_pendaftaran_event_event_ortu (0087).
drop policy if exists "event baca peserta" on public.event;
create policy "event baca peserta" on public.event
for select to authenticated
using (
    status = 'tampil'
    or (select public.is_admin())
    or exists (select 1 from public.pendaftaran_event pe
              where pe.event_id = event.id and pe.ortu_id = (select auth.uid()))
);

insert into public.schema_migrations (versi, nama) values ('0088','rls_initplan')
on conflict (versi) do nothing;

commit;

```

Blok 4–8 mengikuti pola yang sama: `public.is_admin()` → `(select public.is_admin())`, `auth.uid()` → `(select auth.uid())`, tanpa mengubah struktur logika. Yang **tidak** boleh diubah adalah `boleh_lihat_laporan_anak(anak_id)` — argumennya bergantung pada baris, jadi tidak bisa diangkat; itulah sebabnya ia ditaruh paling akhir dalam rantai OR dan mengapa index `pendaftaran_konsultasi(anak_id)` di D.1 penting.

Checklist uji wajib setelah setiap blok

Ini bagian yang tidak boleh dilewati — policy yang salah tidak menimbulkan error, ia diam-diam memperlihatkan atau menyembunyikan data.

- 1. **Ortu A** melihat anaknya sendiri, dan **tidak** melihat anak Ortu B (uji dengan menukar id di URL, bukan hanya lewat UI).
- 2. **Ortu A** melihat riwayat main anaknya di `/anak/<id>/laporan`.
- 3. **Admin** melihat semua anak, semua pendaftaran, semua transaksi.
- 4. **Guru** melihat peserta event yang ditugaskan, dan **tidak** melihat data keuangan.
- 5. **Investor** melihat ringkasan keuangan, dan **tidak** melihat data pribadi anak.
- 6. Anak menyelesaikan satu game → baris `hasil_main` masuk (menguji policy INSERT, yang paling mudah rusak saat menggabungkan policy SELECT).
- 7. `explain (analyze)` pada query `hasil_main` satu anak → cari `InitPlan` di rencana query, dan pastikan tidak muncul `SubPlan` yang dieksekusi ribuan kali.

Aspek	Isi
Dampak	Pada tabel besar, biaya evaluasi policy turun dari O(baris) menjadi O(1) untuk bagian fungsi. Penggabungan 3 policy <code>hasil_main</code> juga memotong dua evaluasi per baris.
Effort	½ hari termasuk uji 4 peran.
Risiko	Tertinggi di Bab D — ini kode keamanan. Kerjakan per blok, uji per blok. Simpan definisi policy lama (<code>select polname, qual from pg_policies where tablename='hasil_main'</code>) sebelum mengubah, supaya bisa dikembalikan.
Prioritas	P0 untuk blok 1–5 · P1 untuk 6–8 · P2 untuk ~150 policy sisanya (hanya bila tabelnya > 1.000 baris).

D.3 Agregasi: pindah ke SQL — sekaligus memperbaiki bug angka salah

D.3.a Ini bukan optimasi. Ini perbaikan kebenaran.

Pola berikut terulang di seluruh modul keuangan dan analitik:

```
// lib/data/keuangan.ts:28-34 – ambilSemua(): SELURUH tabel, tanpa filter tanggal, tanpa limit
const { data } = await s.from('transaksi_keuangan').select('arah,kategori,jumlah,tanggal');
// lalu belasan .filter().reduce() di JavaScript
```

Ada **dua** kerusakan di sini, dan yang kedua jauh lebih serius:

- 1. **Biaya**: `admin/keuangan/laporan` memanggil `getPerBulan()` + `getPerKategori('masuk')` + `getPerKategori('keluar')`, dan ketiganya memanggil `ambilSemua()` → **3× full scan identik dalam satu**

render. `anggaran` melakukannya 2x. `kpi.ts` menarik 4 tabel penuh termasuk **seluruh** `profiles` .

2. **Kebenaran:** PostgREST punya batas baris default **1.000**. Query tanpa `.limit()` **tidak error** saat melewatinya — ia hanya mengembalikan 1.000 baris pertama. Jadi begitu `transaksi_keuangan` mencapai 1.001 baris, total pemasukan di dashboard **salah**, dan tidak ada satu pun tanda di UI, log, maupun Sentry. Angka yang salah tapi terlihat masuk akal lebih berbahaya daripada halaman error.

Lakukan ini sekarang, sebelum apa pun (read-only, 5 menit) — jalankan di SQL Editor:

```
-- Apakah bug pemotongan 1.000 baris SUDAH aktif di produksi?
select 'transaksi_keuangan' t, count(*) n, count(*) > 1000 as sudah_terpotong from
public.transaksi_keuangan
union all select 'profiles',          count(*), count(*) > 1000 from public.profiles
union all select 'anak',              count(*), count(*) > 1000 from public.anak
union all select 'hasil_main',        count(*), count(*) > 1000 from public.hasil_main
union all select 'aktivitas',         count(*), count(*) > 1000 from public.aktivitas
union all select 'pendaftaran_event', count(*), count(*) > 1000 from public.pendaftaran_event
union all select 'langganan',         count(*), count(*) > 1000 from public.langganan
order by n desc;
```

Kolom `sudah_terpotong = true` pada baris mana pun berarti **halaman yang membaca tabel itu sudah menampilkan angka yang salah hari ini**. Untuk `hasil_main` , `true` berarti rencana & streak sebagian anak sudah salah hitung.

D.3.b Presedennya sudah ada di repo ini

`laporan_engagement()` (`0040_laporan_rpc.sql`) sudah melakukan hal yang benar — `SECURITY DEFINER` , guard `is_admin()` , agregasi di dalam database, hanya angka yang keluar. Komentar di baris pertamanya bahkan menuliskan alasannya: "*hindari tarik semua baris ke app*". Yang perlu dilakukan hanyalah **menerapkan pola yang sudah Anda tulis sendiri** ke keuangan dan analitik.

Artefak: `supabase/migrations/0089_ringkas_rpc.sql`

```
-- 0089_ringkas_rpc.sql – agregasi pindah ke SQL. Mengembalikan ANGKA, bukan baris.
-- Ini memperbaiki dua hal sekaligus: (1) 3x full scan per render, (2) bug pemotongan 1.000 baris.
-- Pola mengikuti 0040_laporan_rpc.sql: SECURITY DEFINER + guard peran + STABLE.
```

```
begin;
```

```
-- — Ringkasan keuangan dalam rentang tanggal —————
-- Mengganti: keuangan.ts ambilSemua() + getDashboardKeuangan + getPerBulan + getPerKategori
-- Satu pemanggilan menggantikan 3 full scan. Rentang tanggal WAJIB (bukan opsional) –
-- inilah yang mencegah query ini sendiri menjadi full scan saat tabel membesar.
```

```
create or replace function public.ringkas_keuangan(p_dari date, p_sampai date)
returns json
```

```
language plpgsql stable security definer set search_path = public
as $$
```

```
declare hasil json;
```

```
begin
```

```
  if not (public.is_admin() or public.is_investor()) then
    raise exception 'akses ditolak';
  end if;
```

```
  select json_build_object(
    'dari', p_dari,
    'sampai', p_sampai,
    'total_masuk', coalesce(sum(jumlah) filter (where arah = 'masuk'), 0),
    'total_keluar', coalesce(sum(jumlah) filter (where arah = 'keluar'), 0),
    'laba', coalesce(sum(jumlah) filter (where arah = 'masuk'), 0)
      - coalesce(sum(jumlah) filter (where arah = 'keluar'), 0),
    'jumlah_transaksi', count(*),
```

```
-- per bulan: satu lintasan, bukan 6 filter berulang di JavaScript
```

```
    'per_bulan', (
      select coalesce(json_agg(x order by x.ym), '[]'::json) from (
        select to_char(tanggal, 'YYYY-MM') as ym,
              sum(jumlah) filter (where arah = 'masuk') as masuk,
              sum(jumlah) filter (where arah = 'keluar') as keluar
        from transaksi_keuangan
        where tanggal between p_dari and p_sampai
        group by 1) x),
```

```
-- per kategori: menggantikan DUA pemanggilan getPerKategori()
```

```
    'per_kategori', (
      select coalesce(json_agg(y order by y.total desc), '[]'::json) from (
        select arah, kategori, sum(jumlah) as total, count(*) as n
        from transaksi_keuangan
        where tanggal between p_dari and p_sampai
        group by 1, 2) y)
  ) into hasil
```

```
  from transaksi_keuangan
  where tanggal between p_dari and p_sampai;
```

```
  return hasil;
```

```
end;
```

```
$$;
```

```
revoke all on function public.ringkas_keuangan(date, date) from public, anon;
```

```
grant execute on function public.ringkas_keuangan(date, date) to authenticated;
```

```
-- — Ringkasan analitik N hari terakhir —————
```

```
-- Mengganti: admin/analitik/page.tsx:29-39 (9 query paralel, 2 di antaranya SELURUH tabel)
```

```
-- DAU/WAU/MAU dihitung dengan count(distinct) di DB, bukan dengan Set di memori Node.
```

```
create or replace function public.ringkas_analitik(p_hari int default 30)
```

```
returns json
```

```
language plpgsql stable security definer set search_path = public
```

```
as $$
```

```
declare hasil json; sejak timestampz := now() - make_interval(days => p_hari);
```

```
begin
```

```

if not public.is_admin() then raise exception 'akses ditolak'; end if;

select json_build_object(
  'hari', p_hari,
  'total_ortu', (select count(*) from profiles),
  'total_anak', (select count(*) from anak),
  'ortu_baru', (select count(*) from profiles where created_at >= sejak),
  'dau', (select count(distinct ortu_id) from aktivitas where dibuat_at >= now() - interval '1
day'),
  'wau', (select count(distinct ortu_id) from aktivitas where dibuat_at >= now() - interval '7
days'),
  'mau', (select count(distinct ortu_id) from aktivitas where dibuat_at >= sejak),
  'sesi_main', (select count(*) from hasil_main where tanggal >= sejak::date),
  'anak_aktif', (select count(distinct anak_id) from hasil_main where tanggal >= sejak::date),
  'per_fitur', (
    select coalesce(json_agg(z order by z.n desc), '[]'::json) from (
      select fitur, count(*) n, count(distinct ortu_id) pengguna
      from aktivitas where dibuat_at >= sejak group by 1) z),
  'pendaftaran_event', (select count(*) from pendaftaran_event where created_at >= sejak),
  'pesanan', (select count(*) from pesanan where created_at >= sejak)
) into hasil;

return hasil;
end;
$$;
revoke all on function public.ringkas_analitik(int) from public, anon;
grant execute on function public.ringkas_analitik(int) to authenticated;

-- — Perbaikan laporan_engagement(): 3 full scan → 1 —————
-- Versi lama (0040:8-35) melakukan count+sum, lalu DUA subquery group-by-order-limit-1
-- atas seluruh hasil_main. Versi ini satu lintasan dengan window function.
create or replace function public.laporan_engagement()
returns json
language plpgsql stable security definer set search_path = public
as $$
declare hasil json;
begin
  if not public.is_admin() then raise exception 'akses ditolak'; end if;

  with dasar as (
    select count(*)::bigint as total_sesi,
           coalesce(sum(durasi_detik), 0)::bigint as total_detik
    from hasil_main
  ),
  top_mesin as (
    select mesin from hasil_main group by mesin order by count(*) desc limit 1
  ),
  top_tema as (
    select tema_id from hasil_main where tema_id is not null
    group by tema_id order by count(*) desc limit 1
  )
  select json_build_object(
    'total_sesi', (select total_sesi from dasar),
    'total_detik', (select total_detik from dasar),
    'mesin_populer', (select mesin from top_mesin),
    'tema_populer', (select tema_id from top_tema)
  ) into hasil;

  return hasil;
end;
$$;

insert into public.schema_migrations (versi, nama) values ('0089','ringkas_rpc')
on conflict (versi) do nothing;

```

```
commit;
```

D.3.c Sisi TypeScript

Reader menjadi tipis, dan kolom `arah / jumlah` tidak pernah lagi meninggalkan database:

```
// lib/data/keuangan.ts – pengganti ambilSemua(). Rentang tanggal WAJIB diberikan pemanggil.
export async function ringkasKeuangan(dari: string, sampai: string) {
  const s = await createClient();
  const { data, error } = await s.rpc('ringkas_keuangan', { p_dari: dari, p_sampai: sampai });
  if (error) { log('error', 'db.error', { konteks: 'ringkas_keuangan', pesan: error.message }); throw
  new Error(error.message); }
  return data as RingkasKeuangan;
}
```

Halaman yang berubah: `admin/keuangan/page.tsx`, `.../laporan`, `.../anggaran`, `.../kpi`, `.../insight`, `.../pajak`, `admin/analitik/page.tsx`, `investor/page.tsx`. Semuanya sekarang **wajib** menyebut rentang tanggal — default yang disarankan: bulan berjalan untuk dashboard, 6 bulan untuk laporan, 12 bulan untuk KPI.

Catatan penting saat mengerjakan: setelah beralih ke RPC, angka di dashboard **akan berubah** bila tabel sudah melewati 1.000 baris. Itu **bukan bug baru** — itu angka yang benar muncul untuk pertama kalinya. Catat perbandingan sebelum/sesudah sebagai bukti, dan beri tahu pemilik agar tidak dikira regresi.

D.3.d Kapan naik ke tabel ringkasan harian

RPC di atas cukup sampai ratusan ribu baris. Setelah itu, `count(distinct ortu_id)` atas `aktivitas` yang berisi puluhan juta baris menjadi mahal walaupun ter-index. Tangga berikutnya, **jangan dikerjakan sekarang**:

Tahap	Pendekatan	Pemicu
Sekarang → T1	RPC langsung (di atas)	—
T2	Tabel rollup <code>ringkasan_harian(tanggal, metrik, nilai)</code> diisi <code>pg_cron</code> tiap malam; halaman analitik membaca rollup, bukan tabel mentah	<code>aktivitas</code> > 5 juta baris atau RPC analitik > 2 detik
T2–T3	Retensi: <code>delete from aktivitas where dibuat_at < now() - interval '180 days'</code> setelah dirollup	ukuran DB > 6 GB
T3	Partisi deklaratif bulanan pada <code>hasil_main</code> & <code>aktivitas</code>	insert > 50/detik

Aspek	Isi
Dampak	Memperbaiki angka yang salah (nilai utama). Ditambah: egress DB untuk satu render halaman laporan turun dari "seluruh tabel × 3" menjadi ~2 KB, dan waktu CPU Node untuk mengurai & menjumlahkan hilang sepenuhnya.
Effort	1,5 hari (SQL ½ hari, mengganti 8 pemanggil + verifikasi angka 1 hari).
Risiko	Angka berubah (lihat catatan di atas — ini yang benar). Guard peran di dalam RPC harus diuji: pastikan akun ortu biasa memanggil RPC ini mendapat akses ditolak , bukan data.
Prioritas	P0 — satu-satunya butir P0 di dokumen ini yang memperbaiki kebenaran data, bukan kecepatan atau biaya.

6. Bab E — Pengiriman ke HP: bundle, payload, caching

Pengguna platform ini adalah orang tua dan anak dengan HP kelas menengah-bawah di jaringan Indonesia. Untuk mereka, **byte lebih mahal daripada milidetik CPU server**. Bab ini soal mengurangi apa yang harus diunduh dan diproses HP.

E.1 Code-splitting mesin game

Masalahnya

`GameRunner.tsx:5-20` meng-import **15 mesin game + komponen Reward secara statis**, lalu memilih satu lewat rantai `if/else` saat runtime (`:72-86`). Konsekuensinya: anak yang memainkan satu game tetap mengunduh kode ke-15 mesin. `dynamic()` dan `React.lazy()` **tidak dipakai sama sekali** di seluruh repo — jadi tidak ada satu pun pemisahan kode manual.

Rantai import ini juga menular: `MenuAnak.tsx` (satu-satunya pemakai `GameRunner`) adalah komponen client, sehingga seluruh subtree mesin masuk ke bundle rute `/main/[anakId]` — halaman yang paling sering dibuka anak.

Total sumber mesin $\approx$ 1.832 baris, dengan yang terbesar `MewarnaiGame` (201 baris, memakai `DOMParser` + manipulasi DOM SVG), `IngatanGame` , `HitungBendaGame` , `JiplakGame` .

Artefak: peta mesin dinamis

```
// src/components/game/mesin.ts – SATU tempat pemetaan mesin → loader.
// Menggantikan 15 import statis di GameRunner.tsx:5-20 dan rantai if/else di :72-86.
import dynamic from 'next/dynamic';
import type { ComponentType } from 'react';
import type { Mesin } from '@lib/game/tipe';

// Semua mesin SSR:false – game hanya berjalan setelah interaksi anak, tidak perlu HTML server.
// `loading` menampilkan rangka sederhana; JANGAN spinner berputar (bikin anak menunggu terasa lama).
const opsi = { SSR: false, loading: () => <div className="kp-kartu" style={{ minHeight: 260 }} /> };

export const MESIN: Record<Mesin, ComponentType<any>> = {
  'tekan-sesuai': dynamic(() => import('./ManaYa'), opsi),
  'seret-wadah': dynamic(() => import('./BeresBeres'), opsi),
  'cari-pasangan': dynamic(() => import('./CariPasangan'), opsi),
  'mewarnai': dynamic(() => import('./MewarnaiGame'), opsi),
  'dekode': dynamic(() => import('./Dekode'), opsi),
  'urutan': dynamic(() => import('./UrutanGame'), opsi),
  'jalur': dynamic(() => import('./JalurGame'), opsi),
  'hitung': dynamic(() => import('./HitungGame'), opsi),
  'cocokkan': dynamic(() => import('./CocokkanGame'), opsi),
  'ejakata': dynamic(() => import('./EjaKataGame'), opsi),
  'garis': dynamic(() => import('./GarisGame'), opsi),
  'sukukata': dynamic(() => import('./SukuKataGame'), opsi),
  'jiplak': dynamic(() => import('./JiplakGame'), opsi),
  'hitung-benda': dynamic(() => import('./HitungBendaGame'), opsi),
  'ingatan': dynamic(() => import('./IngatanGame'), opsi),
};
```

Di `GameRunner.tsx`, rantai `if/else` menjadi satu pencarian — dan **mesin yang tidak dikenal harus tetap ditangani**, karena mesin baru bisa ada di data sebelum kodenya ter-deploy:

```
const Mesin = MESIN[paket.mesin as Mesin];
if (!Mesin) return <PesanRamah>Game ini belum bisa dimainkan di versi ini. Coba game lain ya 😊
</PesanRamah>;
return <Mesin data={paket.butir} onSelesai={selesai} />;
```

Reward tetap statis — ia dipakai setiap kali game selesai, jadi lazy-loading-nya hanya menambah jeda di momen paling membahagiakan.

Aspek	Isi
Dampak	Anak mengunduh 1 mesin, bukan 15. Perkiraan penghematan JS rute <code>/main/[anakId]</code> : 60–75% dari porsi mesin game. Perkiraan ini harus diukur , bukan dipercaya — lihat E.5.
Effort	3–4 jam (peta 1 jam, ubah <code>GameRunner</code> 30 menit, uji 15 mesin 2 jam).
Risiko — wajib diuji	(a) Mainkan ke-15 mesin sampai selesai — <code>SSR:false</code> mengubah waktu mount, dan mesin yang membaca ukuran DOM saat mount (<code>JiplakGame</code> memakai <code>getBoundingClientRect</code> , <code>MewarnaiGame</code> memakai <code>DOMParser</code> + <code>innerHTML</code>) paling rentan. (b) Pastikan <code>loading</code> tidak menggeser layout (CLS) — karena itu <code>minHeight</code> dipatok. (c) Uji di jaringan lambat: chunk mesin diunduh saat game dibuka , jadi pramuat aset di <code>IngatanGame</code> kini bersaing dengan unduhan chunk — verifikasi layar "Menyiapkan kartu..." masih wajar.
Prioritas	P1 — dampak besar, tapi kalah mendesak dari kebenaran data & keamanan.

E.2 Pisahkan query ringan dan berat

Masalahnya

Kolom `jsonb` besar selalu ikut terbawa, bahkan di halaman yang hanya menampilkan judul:

Kolom	Isinya	Ikut terkirim di	Padaahal yang dipakai
<code>paket_aset.butir</code>	seluruh isi game: markup SVG mewarnai, palette, grid jalur	<code>pustaka.ts:23</code> → props client di <code>/main/[anakId]</code> dan <code>/pilih-game/[anakId]</code> ; <code>admin/tema/[id]/page.tsx:16</code>	<code>/pilih-game</code> hanya judul + emoji; daftar admin hanya judul, mesin, rentang usia
<code>kelas_bermain.aktivitas</code> + <code>bahan</code>	array langkah & bahan	<code>favorit.ts:6</code> , <code>riwayat-kelas.ts:5</code> , <code>/api/kelas-bermain</code>	daftar favorit & riwayat hanya butuh judul
<code>event.indikator_perkembangan</code>	array indikator	<code>guru.ts:47</code> untuk semua event	hanya dicek <code>Array.isArray(...).length > 0</code> di JavaScript

Presedennya sudah benar di `publik.ts:42,50` — `select('id,judul,tujuan,usia_min,usia_max,sampul_url')` .
Yang kurang adalah menerapkannya ke jalur internal.

Pola: dua konstanta kolom per modul

```
// lib/data/pustaka.ts – RINGAN untuk daftar, PENUH hanya saat game benar-benar dimainkan.  
const PAKET_RINGAN =  
'id,tema_id,mesin,judul,area_skill,usia_min,usia_max,target_detik,kategori_usia_id';  
const PAKET_PENUH = `${PAKET_RINGAN},butir`; // `butir` = isi game, hanya untuk GameRunner
```

Aturan yang berlaku ke depan: **butir** hanya boleh ikut ketika game akan dimainkan pada render itu.
Untuk halaman pemilihan, kirim yang ringan; ambil **butir** satu paket saja saat anak menekan game —
lewat Server Action `ambilButir(paketId)` atau reader `getPaketPenuh(id)` .

Untuk `event.indikator_perkembangan` , hentikan menarik jsonb hanya untuk mengecek panjangnya.
Pindahkan pengecekan ke database sebagai kolom hasil hitung:

```
-- bagian dari 0090_kolom_ringkas.sql  
-- Generated column: dihitung Postgres, ikut di select biasa, TIDAK menarik isi jsonb-nya.  
-- PRASYARAT: pastikan semua baris berisi array atau null, bukan objek –  
--   select count(*) from event where indikator_perkembangan is not null  
--   and jsonb_typeof(indikator_perkembangan) <> 'array';  
--   harus 0, kalau tidak generated column-nya akan error saat dibuat.  
alter table public.event  
  add column if not exists ada_indikator boolean  
  generated always as (  
    coalesce(jsonb_array_length(coalesce(indikator_perkembangan, '[]'::jsonb)), 0) > 0  
  ) stored;
```

Dengan itu `guru.ts:47` cukup `select('id,judul,tanggal,ada_indikator')`.

Aspek	Isi
Dampak	Halaman <code>/pilih-game</code> berhenti mengirim seluruh isi game semua tema ke HP anak — ini kemungkinan penghematan byte terbesar di seluruh dokumen, dan salah satu yang termurah. Endpoint <code>/api/pustaka</code> (dipakai aplikasi mobile untuk sinkronisasi) ikut mengecil drastis.
Effort	<code>/pilih-game</code> + <code>admin/tema/[id]</code> = 30 menit · <code>favorit</code> / <code>riwayat</code> = 30 menit · <code>guru.ts</code> + generated column = 1 jam · <code>/api/pustaka</code> + <code>/api/kelas-bermain</code> (parameter <code>?ringkas=1</code> , jaga kompatibilitas aplikasi mobile lama) = 2 jam.
Risiko	Kompatibilitas aplikasi mobile: jangan hapus kolom dari respons API yang sudah dipakai versi aplikasi yang sudah ter-install — rilis app store tidak bisa dipaksa. Karena itu API memakai parameter opsional, bukan perubahan bentuk respons.
Prioritas	P0 untuk <code>/pilih-game</code> & <code>admin/tema</code> (30 menit, dampak besar) · P1 sisanya.

E.3 Caching berlapis dan perbaikan middleware

E.3.a Ketidakseimbangan yang ada sekarang

5 `unstable_cache` melawan **160** `revalidatePath` di 30 berkas. Artinya invalidasi jauh lebih agresif daripada caching-nya — dan `revalidatePath('/')` yang dipanggil dari mana saja membatalkan seluruh cabang rute, termasuk halaman yang tidak berkaitan dengan perubahan itu.

E.3.b Tiga fase, dari termurah

Fase	Tindakan	Effort	Prio
1	Halaman publik jadi static/ISR. <code>/artikel</code> dan <code>/artikel/[slug]</code> datanya sudah ter-cache lewat <code>unstable_cache</code> , tapi halamannya tetap SSR karena memanggil <code>createClient()</code> (yang menyentuh <code>cookies()</code>). Pakai client anon tanpa cookie di jalur publik (pola <code>publik.ts:7-9</code> sudah ada) + <code>export const revalidate = 300</code> + <code>generateStaticParams</code> untuk artikel.	3 jam	P1
2	Perluas taksonomi tag. Sekarang hanya <code>katalog</code> dan <code>artikel</code> . Pecah menjadi <code>katalog:event</code> , <code>katalog:produk</code> , <code>katalog:kelas</code> , <code>tema</code> , <code>pustaka</code> — supaya menyimpan satu produk tidak membatalkan cache event.	1 hari	P1
3	Ganti <code>revalidatePath</code> bertumpuk dengan <code>updateTag</code>. Mulai dari 5 berkas terbanyak: <code>keuangan-actions.ts</code> (19), <code>admin-event-actions.ts</code> (14), <code>admin-konten.ts</code> (13), <code>admin-bisnis.ts</code> (10), <code>psikolog-actions.ts</code> (9). Jangan semua 160 sekaligus.	1–2 hari	P1

E.3.c Middleware: dua perbaikan terpisah

Perbaikan 1 — matcher (15 menit, P0, tanpa risiko). Matcher sekarang (`proxy.ts:53`) tidak mengecualikan `.css`, `.js`, `.woff2`, `sitemap.xml`, `robots.txt`, `_next/data`. Setiap sub-resource itu memicu invocation middleware **dan** satu `auth.getUser()` ke Supabase:

```
// src/proxy.ts — matcher yang benar
export const config = {
  matcher: [
    // Kecualikan: API (pakai Bearer), aset build, berkas statis, dan metadata SEO.

    '/(?!(?!\api|_next/static|_next/image|_next/data|favicon.ico|logo.png|sitemap.xml|robots.txt|llms.txt|manif
(?:png|jpg|jpeg|svg|ico|webp|css|js|woff2?|ttf|map|xml|txt)$).*')',
  ],
};
```

Perbaikan 2 — kurangi `auth.getUser()` (3–4 jam, P1, butuh prasyarat). Setiap navigasi memicu satu perjalanan jaringan ke Supabase Auth; pada `/main/[anakId]` bahkan **tiga kali** dalam satu render.

Di sini saya harus berterus terang soal keamanan: **jangan mengganti `getUser()` dengan membaca isi JWT tanpa memverifikasi tanda tangannya**. Token yang tidak diverifikasi bisa dipalsukan, dan di aplikasi ini itu berarti seseorang bisa mengaku sebagai admin. Yang **boleh** adalah verifikasi tanda tangan **secara lokal** memakai kunci publik — itulah gunanya `getClaims()` dengan asymmetric JWT signing keys di Supabase: aman secara kriptografis, tanpa perjalanan jaringan.

Jadi urutannya: **(a)** aktifkan asymmetric JWT signing keys di dashboard Supabase 🔍, **(b)** ganti `auth.getUser()` → `auth.getClaims()` di `proxy.ts`, **(c)** teruskan hasilnya ke halaman agar tidak dipanggil ulang, **(d)** uji bahwa token kedaluwarsa & token dipalsukan sama-sama ditolak. Selama (a) belum tersedia, **biarkan `getUser()`** — 100 ms latensi jauh lebih murah daripada celah autentikasi.

Perbaikan 3 — pindahkan cek menu admin (1 jam, P1). `proxy.ts:31,33` menjalankan 2 query DB untuk setiap navigasi `/admin/*`. Pindahkan ke `admin/layout.tsx` (yang memang sudah memanggil `getAksesAdmin()`), sehingga tidak dieksekusi dua kali, dan bungkus `pengaturan_menu` dengan `unstable_cache` bertag — tabel satu baris yang nyaris tidak pernah berubah.

E.4 Gambar: keputusan yang berlawanan dengan dugaan umum

Ini titik di mana dua analisis dalam penyusunan dokumen ini bertentangan, jadi keputusannya ditulis eksplisit beserta alasannya.

Dugaan wajar: "ada 12 `<img>` mentah, pindahkan semuanya ke `next/image`." Untuk **aset game** itu **keputusan yang salah**, dan berpotensi lebih mahal daripada masalah yang diperbaiki.

Alasannya: kuota **Image Optimization** Vercel dihitung per gambar sumber unik yang dioptimasi. `Aset.tsx` adalah jalur terpanas aplikasi dengan **ratusan hingga ribuan aset unik** (setiap kartu game adalah gambar berbeda). Memindahkannya ke `next/image` berarti menukar egress Supabase yang murah dan berkuota besar dengan kuota transformasi Vercel yang jauh lebih sempit — lalu tagihan muncul di tempat baru.

Yang dilakukan untuk aset game (lebih murah, dampak lebih besar):

#	Tindakan	Mengapa berdampak	Effort
1	Set <code>cacheControl: '31536000'</code> saat unggah (sekarang tidak pernah diset → default pendek). Nama berkas sudah unik per unggahan, jadi isinya immutable dan aman di-cache setahun.	Aset berhenti diunduh ulang setiap kali anak membuka game. Ini pengungkit egress terbesar di seluruh dokumen, dan pekerjaannya 20 menit.	20 menit
2	Turunkan preset kompresi aset game dari <code>maksDim 512</code> (<code>AsetInput.tsx:22</code>) → 256 px . Perhitungannya: aset dirender pada 42–90 px CSS (terbesar <code>size={90}</code>), jadi pada HP dengan DPR 3 dibutuhkan hingga 270 px — 256 px menutup hampir semuanya, sementara 192 px akan tampak kabur pada aset terbesar . Jangan turunkan lebih jauh dari ini.	Luas piksel turun 4× ⇒ byte per aset turun ~4×, dan waktu decode di HP kelas bawah ikut turun.	1 jam + backfill
3	Tetap <code></code> , tapi tambahkan <code>width / height</code> eksplisit agar tidak ada pergeseran layout.	—	15 menit

Yang memang layak `next/image` : `Sampul.tsx` (jumlah tema/kelas sedikit → varian optimasi sedikit), dan gambar konten seperti banner event, foto produk, sampul artikel — beberapa di antaranya sudah memakainya.

Yang harus TETAP `<img>` selain aset game: `BuktiLightbox`. Setelah bukti bayar pindah ke signed URL (F.3), URL-nya berubah setiap kali dibuat — dan setiap URL unik akan dihitung sebagai gambar baru oleh optimizer. Ini alasan teknis, bukan preferensi gaya.

E.5 Pasang alat ukur SEBELUM mengerjakan bab ini

`@vercel/analytics` sudah terpasang di `layout.tsx:58`. Aktifkan **Web Vitals** di dashboard Vercel (dan Speed Insights bila plan mengizinkan 🔍) **sebelum** memulai E.1–E.4.

Alasannya sederhana: seluruh angka penghematan di bab ini adalah perkiraan saya dari membaca kode, bukan pengukuran pada HP Indonesia sungguhan. Tanpa data sebelum-sesudah, pekerjaan ini akan selesai tanpa ada yang tahu apakah ia berhasil — dan kalau ternyata bottleneck sebenarnya ada di tempat lain (misalnya TTFB dari SSR, bukan ukuran bundle), Anda baru mengetahuinya setelah membayar biaya kerjanya.

Effort 30 menit. Prioritas P0 — bukan karena mendesak, tapi karena ia prasyarat agar sisa bab ini bisa dievaluasi.

7. Bab F — Storage, egress & kontrol biaya

F.1 Satu pintu unggahan: `src/lib/upload.ts`

Empat masalah berbeda punya satu solusi, jadi dikerjakan sekaligus:

- `UploadNota.tsx:17-27` punya fungsi kompresi **duplikat** yang tidak memakai `lib/img.ts`.
- Duplikat itu **melempar error** untuk masukan non-raster, sementara `lib/img.ts` punya fallback yang benar. Akibat nyata: pada iPhone dengan foto HEIC, unggah nota **gagal**.
- Tidak ada validasi ukuran di klien** sama sekali, di 12 titik unggah.

4. `cacheControl` tidak pernah diset (lihat [E.4](#)).

```

// src/lib/upload.ts – SATU pintu untuk semua unggahan ke Storage.
// Menggantikan kompresWebp() duplikat di UploadNota.tsx dan 12 pemanggilan .upload() tersebar.
'use client';
import type { SupabaseClient } from '@supabase/supabase-js';
import { kompresGambar } from '@lib/img';

export const BATAS = {
  gambar: 5 * 1024 * 1024, // 5 MB sebelum kompresi
  svg: 256 * 1024, // SVG tidak dikompres → batas ketat
  pdf: 10 * 1024 * 1024,
} as const;

export const MIME_GAMBAR = ['image/jpeg', 'image/png', 'image/webp', 'image/svg+xml'];
export const MIME_DOK = ['application/pdf'];

/** Preset per jalur pakai. `maksDim` menentukan berapa byte yang akhirnya diunduh HP. */
export const PRESET = {
  aset: { maksDim: 256, kualitas: 0.78 }, // aset game: tampil 42-90px CSS, 256px menutup DPR 3
  sampul: { maksDim: 480, kualitas: 0.80 }, // sampul tema/kelas/produk
  banner: { maksDim: 1080, kualitas: 0.82 }, // banner event, tampil lebar penuh
  bukti: { maksDim: 1280, kualitas: 0.78 }, // bukti bayar/nota: harus tetap terbaca
  cetak: null, // template sertifikat & stiker: JANGAN dikompres
} as const;

/** Pesan Bahasa Indonesia bila tidak lolos, null bila lolos. */
export function validasiFile(file: File, jenis: 'gambar' | 'dok'): string | null {
  const mime = file.type || '';
  const daftar = jenis === 'gambar' ? MIME_GAMBAR : MIME_DOK;
  if (!daftar.includes(mime)) {
    if (mime === 'image/gif') return 'Format GIF tidak didukung. Pakai JPG, PNG, atau WebP ya.';
    if (mime === 'image/heic' || mime === 'image/heif')
      return 'Foto HEIC dari iPhone belum didukung. Di iPhone: Pengaturan → Kamera → Format → "Paling Kompatibel", lalu foto ulang.';
    if (!mime) return 'Jenis berkas tidak terdeteksi. Coba pilih ulang dari Galeri.';
    return `Jenis berkas ${mime} tidak didukung.`;
  }
  const batas = mime === 'image/svg+xml' ? BATAS.svg : jenis === 'dok' ? BATAS.pdf : BATAS.gambar;
  if (file.size > batas) {
    const mb = (batas / 1024 / 1024).toFixed(batas < 1024 * 1024 ? 2 : 0);
    return mime === 'image/svg+xml'
      ? `SVG maksimal ${mb} MB. Simpan ulang sebagai "Optimized SVG" di Inkscape/Illustrator.`
      : `Berkas maksimal ${mb} MB (berkas ini ${(file.size / 1024 / 1024).toFixed(1)} MB).`;
  }
  return null;
}

export async function unggah(
  sb: SupabaseClient,
  folder: string,
  file: File,
  opsi: { jenis: 'gambar' | 'dok'; preset?: keyof typeof PRESET; bucket?: string },
): Promise<{ path: string; url: string }> {
  const salah = validasiFile(file, opsi.jenis);
  if (salah) throw new Error(salah);

  const bucket = opsi.bucket ?? 'aset';
  const preset = PRESET[opsi.preset ?? 'sampul'];
  const perluKompres = opsi.jenis === 'gambar' && file.type !== 'image/svg+xml' && preset !== null;

  const { blob, ext } = perluKompres
    ? await kompresGambar(file, preset)
    : { blob: file as Blob, ext: (file.name.split('.').pop() || 'bin').toLowerCase() };

```

```

const path = `${folder}/${Date.now()}-${Math.floor(performance.now())}.${ext}`;
const { error } = await sb.storage.from(bucket).upload(path, blob, {
  upsert: false,
  // MIME EKSPISIT – jangan `blob.type || undefined`. Ini prasyarat F.2.
  contentType: blob.type || file.type || 'application/octet-stream',
  // Nama unik per unggahan ⇒ isi immutable ⇒ aman di-cache setahun. Pengungkit egress terbesar.
  cacheControl: '31536000',
});
if (error) throw error;

return { path, url: sb.storage.from(bucket).getPublicUrl(path).data.publicUrl };
}

```

Dua belas titik yang bermigrasi: `components/admin/AsetInput.tsx`, `components/UploadNota.tsx`, `components/UploadDok.tsx`, `app/pesanan/[id]/BuktiUpload.tsx`, `app/event/[id]/daftar/DaftarForm.tsx`, `app/admin/TambahTemaForm.tsx`, `app/admin/event/EventAdmin.tsx` (×3), `app/admin/kelas-bermain/KelasAdmin.tsx` (×2), `app/admin/artikel/[id]/ArtikelForm.tsx`, `app/admin/produk/ProdukAdmin.tsx`.

Aspek	Isi
Dampak	<code>cacheControl</code> setahun (egress aset game turun besar) + preset 256 px (byte per aset turun ~4×) + bug unggah HEIC hilang + validasi ukuran + prasyarat F.2 terpenuhi. Empat perbaikan, satu berkas.
Effort	4–6 jam (tulis 1 jam, migrasi 12 titik + uji tiap form 3–5 jam).
Risiko — wajib diuji	Uji setiap form unggah di HP nyata (Android dan iPhone): bukti bayar pesanan, bukti daftar event, nota keuangan, dokumen sponsor, sampul tema, banner + template sertifikat + background stiker event, worksheet + sampul kelas, sampul artikel, gambar produk, aset game. Perhatian khusus: <code>tools/backfill-util.mjs:4</code> mengecualikan <code>event/sertifikat</code> & <code>event/stiker</code> dari kompresi karena template cetak butuh resolusi tinggi — karena itu ada <code>PRESET.cetak = null</code> ; pastikan dua jalur itu memakainya.
Prioritas	P0 — memuat dua quick win egress terbesar sekaligus menjadi prasyarat F.2 dan F.3.

F.2 Batas bucket

Jangan jalankan SQL di bawah sebelum `src/lib/upload.ts` ter-deploy. Repo memanggil `.upload(..., { contentType: blob.type || undefined })` di 12 titik. Bila `blob.type` kosong — sering terjadi pada pemilih berkas Android dan beberapa kamera — Supabase memakai `application/octet-stream`. Begitu `allowed_mime_types` aktif, unggahan itu **ditolak**: orang tua tidak bisa mengirim bukti bayar dan admin tidak bisa mengunggah gambar produk. Kerugiannya langsung ke pendapatan.

```
-- 0091_storage_kebijakan.sql – batas ukuran & tipe berkas. Idempoten (update, bukan insert).
-- JALANKAN SETELAH src/lib/upload.ts ter-deploy dan diverifikasi di produksi.

-- Lihat kondisi sekarang lebih dulu (read-only):
-- select id, public, file_size_limit, allowed_mime_types from storage.buckets;

update storage.buckets set
  -- 10 MB: cukup untuk worksheet PDF hasil scan; menutup kecelakaan video/RAW.
  -- CATATAN: nilai ini tidak boleh melebihi batas global proyek
  -- (Dashboard → Settings → Storage → Upload file size limit). Cek dulu.
  file_size_limit = 10485760,
  allowed_mime_types = array[
    'image/jpeg', 'image/png', 'image/webp', 'image/svg+xml', 'application/pdf'
  ]
  -- image/gif SENGAJA TIDAK dimasukkan – lihat F.5.
  -- application/octet-stream SENGAJA TIDAK dimasukkan: memasukkannya meniadakan
  -- seluruh gunanya. Perbaiki klien, jangan longgarkan server.
where id = 'aset';
```

Effort 10 menit · Prioritas P1 (P0-nya adalah F.1 yang menjadi prasyaratnya) · **Uji sesudahnya:** unggah dari Android nyata, dari iPhone (HEIC!), gambar produk, worksheet PDF, dan SVG aset game — kelimanya harus berhasil.

F.3 Memindahkan **bukti/** & **nota/** ke bucket privat + signed URL

F.3.a Pernyataan masalah (privasi, bukan performa)

Bucket **aset** dideklarasikan **public = true** (**0007_storage_aset.sql:2-3**) dengan policy baca terbuka untuk siapa pun — bahkan tanpa **to authenticated** . Path bukti berbentuk **bukti/<Date.now()>-<performance.now()>.<ext>** .

Konsekuensi konkretnya:

- **Isi berkasnya:** bukti transfer memuat nama pemilik rekening, nomor rekening, nominal, kadang nomor HP. Nota memuat data vendor dan harga internal.
- **URL absolutnya tersimpan di DB** (**pesanan.bukti_url** , **pendaftaran_event.bukti_url** , **aset.invoice_url** , **transaksi_keuangan.lampiran_url**) lalu mengalir ke log, tangkapan layar WhatsApp, dan riwayat browser. Siapa pun yang pernah melihat satu URL bisa membukanya selamanya, tanpa login.
- **Entropi path rendah:** **Date.now()** dalam jendela yang bisa diperkirakan + **performance.now()** yang berkorelasi. Ini bukan rahasia kriptografis.
- Ini data pribadi menurut **UU PDP** — bukan sekadar kerapian teknis.

Ada pula celah tulis: policy **"aset unggah bukti user"** (**0017_event.sql:47-49**) mengizinkan **setiap** pengguna terautentikasi menulis ke mana pun di bawah **bukti/** , tanpa pembatasan per pengguna.

F.3.b Keputusan arsitektur, dan yang ditolak

Fakta teknis yang harus jelas: pada bucket **public = true** , endpoint **/storage/v1/object/public/...** **tidak memeriksa RLS**. Jadi memperketat policy pada **storage.objects** **tidak akan** membuat **bukti/** privat. Satu-satunya cara adalah memindahkan berkasnya ke bucket dengan **public = false** .

Ditolak: mengubah `aset` sendiri menjadi privat. Itu merusak setiap URL publik yang sudah tersimpan untuk aset game, gambar produk, banner event, dan sampul artikel (ratusan baris DB), sekaligus menambah perjalanan penandatanganan di jalur terpanas (`Aset.tsx`).

Dipilih: bucket kedua bernama `privat` , dan **berhenti menyimpan URL absolut** — simpan **path**, tandatangani saat render.

```
-- 0092_bucket_privat.sql – bucket privat untuk bukti bayar & nota keuangan.
-- Idempoten. TIDAK mengubah bucket `aset` (aset game & produk tetap publik).

insert into storage.buckets (id, name, public, file_size_limit, allowed_mime_types)
values ('privat', 'privat', false, 5242880,
      array['image/jpeg','image/png','image/webp','application/pdf'])
on conflict (id) do update
  set public = false,
      file_size_limit = excluded.file_size_limit,
      allowed_mime_types = excluded.allowed_mime_types;

-- — TULIS: user hanya boleh menulis ke bukti/<uid-nya>/... —————
-- Sekaligus menutup celah policy lama yang mengizinkan tulis ke mana pun di bawah bukti/.
drop policy if exists "privat unggah bukti sendiri" on storage.objects;
create policy "privat unggah bukti sendiri" on storage.objects for insert to authenticated
with check (
  bucket_id = 'privat'
  and (storage.foldername(name))[1] = 'bukti'
  and (storage.foldername(name))[2] = (select auth.uid())::text
);

drop policy if exists "privat unggah admin" on storage.objects;
create policy "privat unggah admin" on storage.objects for insert to authenticated
with check (bucket_id = 'privat' and (select public.is_admin()));

-- — BACA: pemilik + admin + investor (khusus nota). Guru & psikolog TIDAK. —
-- createSignedUrl() tetap melewati policy ini, jadi ini penjaga yang sebenarnya.
drop policy if exists "privat baca" on storage.objects;
create policy "privat baca" on storage.objects for select to authenticated
using (
  bucket_id = 'privat'
  and (
    (select public.is_admin())
    or ((select public.is_investor()) and (storage.foldername(name))[1] = 'nota')
    or ((storage.foldername(name))[1] = 'bukti'
        and (storage.foldername(name))[2] = (select auth.uid())::text)
  )
);

-- — HAPUS: hanya admin. User TIDAK boleh menghapus bukti bayarnya (jejak audit). —
drop policy if exists "privat hapus admin" on storage.objects;
create policy "privat hapus admin" on storage.objects for delete to authenticated
using (bucket_id = 'privat' and (select public.is_admin()));
```

```
// src/lib/data/lampiran.ts – resolver TOLERAN. Mengikuti pola wajib di CLAUDE.md:
// data LAMA (URL absolut) tetap tampil, data BARU (path) ditandatangani saat render.
import { createClient } from '@lib/supabase/server';

/** Terima URL absolut lama ATAU path baru. Kembalikan URL siap pakai di <img src>. */
export async function urlLampiran(nilai?: string | null): Promise<string | null> {
  const v = (nilai ?? '').trim();
  if (!v) return null;
  if (/^https?:\/\//.test(v)) return v; // baris lama: biarkan
  const s = await createClient();
  const { data, error } = await s.storage.from('privat').createSignedUrl(v, 60 * 10);
  if (error) { console.error('signedUrl:', error.message, v); return null; }
  return data.signedUrl;
}
```

Masa berlaku 10 menit cukup untuk admin memeriksa bukti, terlalu pendek untuk dibagikan bermakna. Untuk halaman admin yang lama dibuka, pertimbangkan 60 menit.

F.3.c Jalur migrasi tiga fase

Fase 1 — hentikan pendarahan (P0, ~3 jam). Jalankan `0092`. Ubah `BuktiUpload.tsx`, `DaftarForm.tsx`, `UploadNota.tsx`, `UploadDok.tsx` agar mengunggah ke bucket `privat` dengan folder `bukti/<uid>` atau `nota`, dan menyimpan **path, bukan URL**. Pakai `urlLampiran()` di ~6 titik render (`BuktiLightbox`, detail pesanan admin, detail pendaftaran admin, `getTransaksiDetail`, halaman aset keuangan, halaman pesanan milik ortu). Sejak titik ini **tidak ada bukti baru yang publik**.

Fase 2 — pindahkan yang lama (P1, ~4 jam). Skrip sekali-jalan `tools/pindah-bukti.mjs` dijalankan owner dari laptop dengan service role key (**kunci itu jangan pernah masuk env Vercel**):

```
Untuk setiap baris di pesanan.bukti_url, pendaftaran_event.bukti_url,
aset.invoice_url, transaksi_keuangan.lampiran_url yang berupa URL publik ke aset/bukti|nota:
1. tentukan path tujuan: bukti/<ortu_id>/<namaberkas> | nota/<namaberkas>
2. unduh dari `aset` → unggah ke `privat` (upsert:true, cacheControl:'0')
3. UPDATE kolom DB menjadi path tujuan
4. HAPUS objek sumber di `aset`
Idempoten: baris yang sudah berupa path dilewati.
Urutan 2→3→4 disengaja – bila gagal di tengah, sumber MASIH ADA dan kolom DB masih
menunjuk URL lama yang valid. Paling buruk hanya duplikat, tidak ada bukti yang hilang.
Wajib: --dry-run sebagai default, --yakin untuk benar-benar menulis, log ke berkas.
```

Storage bukan transaksional, jadi jangan membungkusnya dalam transaksi DB — urutan di atas dipilih supaya kegagalan selalu "aman gagal".

Fase 3 — tutup pintunya (P1, 15 menit). Setelah query verifikasi mengembalikan 0 di semua baris:

```

select 'pesanan' t, count(*) from public.pesanan
  where bukti_url like 'http%' and bukti_url like '%/aset/bukti/%'
union all select 'pendaftaran_event', count(*) from public.pendaftaran_event
  where bukti_url like 'http%' and bukti_url like '%/aset/bukti/%'
union all select 'aset', count(*) from public.aset
  where invoice_url like 'http%' and invoice_url like '%/aset/nota/%'
union all select 'transaksi_keuangan', count(*) from public.transaksi_keuangan
  where lampiran_url like 'http%' and lampiran_url like '%/aset/nota/%';

-- Baru setelah semuanya 0:
drop policy if exists "aset unggah bukti user" on storage.objects;
-- Sisa objek di aset/bukti|nota dihapus lewat tools/pindah-bukti.mjs --sapu
-- (JANGAN lewat SQL: menghapus baris storage.objects tanpa menghapus berkasnya
-- meninggalkan berkas hantu yang tetap ditagih).

```

Aspek	Isi
Dampak	Kebocoran data pribadi tertutup. Bonus: bucket privat tidak bisa di-hotlink → egress dari luar hilang; dan celah tulis lintas-pengguna di <code>bukti/</code> tertutup.
Effort	Fase 1 ~3 jam · Fase 2 ~4 jam · Fase 3 ~15 menit → ≈1 hari .
Risiko — wajib diuji	(1) Admin tetap bisa melihat bukti lama (URL absolut) dan baru (path) — uji keduanya. (2) Investor bisa melihat nota tapi tidak bukti bayar ortu. (3) Ortu bisa melihat buktinya sendiri dan tidak milik orang lain — uji dengan menukar path manual. (4) Bila <code>createSignedUrl</code> gagal, UI menampilkan "bukti tidak dapat dibuka", bukan gambar rusak tanpa penjelasan. (5) Signed URL kedaluwarsa saat halaman admin dibuka lama → pertimbangkan 60 menit. (6) Jangan kirim URL bertanda tangan ke <code>next/image</code> .
Prioritas	Fase 1 P0 (privasi) · Fase 2–3 P1 .

F.4 Berkas orphan

Semua unggahan memakai nama berbasis `Date.now()` dengan `upsert: false`, dan **tidak ada satu pun jalur yang menghapus berkas lama**. Ganti banner event 10× → 10 berkas, 1 dirujuk. Lebih buruk: `EventAdmin.tsx:46` mengunggah **saat berkas dipilih**, sebelum form disimpan — admin yang membatalkan form tetap meninggalkan berkas.

⚠ Bahaya terbesar justru dari pembersihannya sendiri

Aset game dirujuk dari **DALAM** jsonb `paket_aset.butir`, bukan dari kolom `*_url`. Skrip pembersih naif yang berlogika "hapus objek yang tidak ada di kolom `*_url` mana pun" akan **MENGHAPUS SELURUH ASET GAME** dan merusak setiap paket mewarnai, cocokkan, dan ingatan. Ini kesalahan yang tidak bisa dibatalkan.

Ekstraksi rujukan yang benar dari jsonb — **jalankan dan bandingkan jumlahnya sebelum mempercayai skrip pembersih apa pun**:

```
-- Semua path Storage yang tertanam di paket_aset.butir, rekursif, kedalaman berapa pun.
select count(*) as rujukan_dalam_butir from (
  select distinct trim(both '' from v::text) as ref
  from public.paket_aset p,
       lateral jsonb_path_query(p.butir, '$.**') v
  where jsonb_typeof(v) = 'string'
        and (v #>> '{}') like '%/storage/v1/object/%'
) x;
```

Sumber rujukan lengkap yang harus dikumpulkan skrip: `tema.sampul` · `paket_aset.butir` (jsonb, rekursif) · `kelas_bermain.sampul_url` · `kelas_bermain.worksheet_url` · `kelas_bermain.bahan[].link` · `event.gambar_url` · `template.sertifikat & stiker event` · `produk.gambar_url` · `artikel.sampul` · `pesanan.bukti_url` · `pendaftaran_event.bukti_url` · `aset.invoice_url` · `transaksi_keuangan.lampiran_url`.

Rancangan `tools/bersihkan-orphan.mjs` (P1, ~4 jam):

- Kumpulkan `SET_RUJUKAN` dari semua sumber di atas (normalisasi URL → path); `list()` bucket rekursif → `SET_OBJEK`.
- Kandidat = `SET_OBJEK - SET_RUJUKAN`, **dan** umur objek > **30 hari**. Masa tenggang ini menutup kasus nyata "diunggah saat memilih berkas, form disimpan besok".
- Default `--dry-run`: cetak jumlah, total byte, dan **20 contoh path** per folder. Butuh `--yakin` untuk menghapus. Selalu tulis log ke berkas.
- **Palang pengaman**: bila kandidat > 30% dari total objek, **berhenti** dan minta konfirmasi eksplisit — itu tanda kuat satu sumber rujukan terlewat.
- Ikuti preseden `tools/backfill-kompres.mjs` + `backfill-util.mjs`: taruh logika keputusan di fungsi murni `bolehHapus(path, umurHari, setRujukan)` dan **tulis unit test-nya**, termasuk kasus yang membuktikan aset yang dirujuk dari `butir` **tidak** boleh dihapus.

Menghentikan pertumbuhannya (P1–P2):

Pendekatan	Cara	Catatan
Hapus-saat-ganti (P1, ~4 jam)	Server Action yang menyimpan URL baru tahu nilai lamanya → hapus setelah update DB berhasil	Urutan penting: hapus setelah commit. Kalau dibalik dan update gagal, baris DB menunjuk berkas yang sudah tidak ada.
Path ber-entitas + <code>upsert:true</code> (P2, ~1 hari)	<code>produk/<produk_id>.webp</code> , <code>event/<event_id>.webp</code> — mengganti gambar = menimpa ⇒ no orphan, selamanya	URL tidak berubah, jadi cache setahun butuh pembatal: pakai <code>?v=<updated_at epoch></code> pada URL yang dirender. Satu-satunya perbaikan yang benar-benar menutup masalah alih-alih menyapunya berkala.

Ukur dulu sebelum memutuskan seberapa mendesak:

```
select (storage.foldername(name))[1] as folder, count(*) jml,
       pg_size_pretty(sum((metadata->>'size')::bigint)) besar
from storage.objects where bucket_id = 'aset' group by 1 order by sum((metadata->>'size')::bigint) desc;
```

Risiko: penghapusan tidak bisa dibatalkan. Palang pengaman + dry-run + masa tenggang 30 hari + unit test rujukan-jsonb adalah syarat mutlak, bukan opsi. Ini pekerjaan paling berisiko di seluruh dokumen —

perlakukan sesuai.

F.5 SVG, GIF, dan PDF yang lolos tanpa kompresi

`lib/img.ts:18-20` sengaja melewati `image/svg+xml` dan `image/gif`; PDF worksheet tidak pernah tersentuh.

Format	Keputusan	Alasan	Prio
SVG	Batas keras 256 KB di klien (<code>BATAS.svg</code>) + sanitasi sebelum disimpan dan sebelum dirender	SVG dari Inkscape/Illustrator rutin 10–20× lebih besar dari perlunya karena metadata; menolak + memberi instruksi lebih murah daripada memelihara optimizer. Sanitasi bukan soal ukuran: MewarnaiGame memakai <code>DOMParser</code> + <code>innerHTML</code> atas markup dari <code>paket_aset.butir</code> . <code>innerHTML</code> tidak menjalankan <code><script></code> , tetapi <code><svg onload=...></code> / <code><image onerror=...></code> tetap menyala. Jangkauannya terbatas ke admin (hanya admin menulis <code>butir</code>), jadi P1 bukan P0 — tapi jangan dibiarkan. Cukup ~30 baris: buang node <code>script</code> / <code>foreignObject</code> / <code>use[href^=http]</code> dan semua atribut berawalan <code>on</code> . Tanpa dependency.	P1
GIF	Tolak (tidak masuk <code>allowed_mime_types</code> + pesan ramah di <code>validasiFile</code>)	Rasio byte/kualitas terburuk dan tidak pernah dibutuhkan sebagai aset game statis atau foto produk. Menolak lebih baik daripada mengonversi diam-diam ke frame pertama.	P1
PDF worksheet	Batas keras 10 MB + <code>file_size_limit</code> sebagai jaring server. Jangan tambah library kompresi PDF . Sajikan sebagai tautan unduh, bukan <code><embed></code> inline	Kompresi PDF di klien butuh <code>pdf-lib</code> /wasm Ghostscript — puluhan hingga ratusan KB dependency dan CPU berat, di aplikasi yang penggunanya HP kelas bawah, untuk memperbaiki alur yang hanya dipakai admin . Yang menyelamatkan egress adalah tidak mengembed PDF di halaman yang dibuka setiap orang tua.	P1

F.6 Estimasi egress & kontrol biaya

F.6.a Asumsi (koreksi angka ini bila data nyata berbeda)

Kode	Asumsi	Skenario "sekarang"	Skenario "5×"
A1	Keluarga aktif/bulan	200	1.000
A3	Sesi bermain/anak/bulan	12	12
A5	Aset gambar per layar game	12	12
A6	Ukuran aset sebelum perbaikan (WebP 512 px)	60 KB	60 KB
A7	Ukuran aset sesudah (preset 256 px — luas 4× lebih kecil)	15 KB	15 KB
A8/A9	Cache hit aset sebelum → sesudah	~0% → ~85%	sama
A10	Gambar konten per halaman non-game	150 KB	150 KB

F.6.b Hitungan egress Storage

Sesi/bulan = $200 \times 12 = \mathbf{2.400}$ (skenario 5×: 12.000).

	Sekarang	Sesudah F.1 + E.4
Aset game	$2.400 \times 12 \times 60 \text{ KB} \approx \mathbf{1,73 \text{ GB}}$	$2.400 \times 12 \times 15 \text{ KB} \times 0,15 \approx \mathbf{65 \text{ MB}}$
Gambar konten	$\approx \mathbf{0,24 \text{ GB}}$	$\approx \mathbf{0,04 \text{ GB}}$
Total	$\approx \mathbf{2,0 \text{ GB/bulan}}$	$\approx \mathbf{0,1 \text{ GB/bulan}} (\approx 20\times \text{ turun})$
Skenario 5×	$\approx \mathbf{10 \text{ GB/bulan}} \rightarrow \text{melewati batas Supabase Free}$	$\approx 0,5 \text{ GB/bulan} \rightarrow \text{aman}$

Kesimpulannya: **egress Storage belum jadi masalah hari ini, tapi akan melewati batas tepat saat pemasaran mulai berhasil**. Dua quick win di F.1 memindahkan tebing itu sekitar 20× dengan biaya kerja beberapa jam.

F.6.c Egress database — yang sering terlupakan

Ini konsekuensi langsung dari agregasi-di-JavaScript (D.3), dan pada skala besar **melampaui** egress gambar:

Halaman	Sekarang	Sesudah RPC
<code>/admin/keuangan/laporan</code>	3 × seluruh <code>transaksi_keuangan</code>	1 × ~2 KB
<code>/admin/keuangan/anggaran</code>	2 × seluruh <code>transaksi_keuangan</code>	1 × ~2 KB
<code>/admin/analitik</code>	seluruh <code>anak</code> + seluruh <code>profiles</code> + 30 hari <code>hasil_main</code> + <code>aktivitas</code>	1 × ~3 KB
<code>/admin/keuangan/kpi</code>	seluruh <code>langganan</code> + <code>pembayaran_langganan</code> + <code>transaksi_keuangan</code> + <code>profiles</code>	1 × ~3 KB

Pada 200 pengguna angkanya kecil. Pada **20.000 pengguna**, `profiles` sendiri $\approx 4 \text{ MB}$; admin membuka halaman itu 30×/bulan × 3 scan $\Rightarrow \mathbf{\sim 360 \text{ MB/bulan}}$ untuk satu halaman, ditambah waktu CPU Node untuk mengurainya. Jadi RPC di Bab D adalah pekerjaan **biaya**, bukan hanya kecepatan.

F.6.d Risiko biaya, diurut dari yang paling nyata

1. **Kuota Image Optimization Vercel** — akan meledak bila `Aset.tsx` dipindah ke `next/image`. Cara mencegahnya: **jangan lakukan itu** (E.4). Ini risiko yang diciptakan oleh "perbaikan" yang salah, bukan oleh kondisi sekarang.
2. **Compute & disk Supabase** saat `aktivitas` tumbuh (1 insert per page view, 3 index). Aturan retensi 180 hari lebih berdampak pada biaya daripada optimasi query apa pun di tabel itu — **P1, dan sangat murah**.
3. **Egress database** dari query tanpa `limit` (F.6.c).
4. **Egress Storage** — nomor 4, bukan nomor 1. Penting ditulis agar prioritas tidak salah arah.
5. **Bukan teknis, tapi paling mendesak**: Vercel Hobby melarang penggunaan komersial (§0.3).

Catatan wajib: kuota & harga Vercel dan Supabase berubah beberapa kali per tahun. Jangan memperlakukan angka di dokumen ini sebagai fakta abadi — yang stabil dan berguna adalah **asumsi**

A1–A10 dan formulanya, yang bisa dihitung ulang kapan pun kuotanya berubah.

F.6.e Budget alert & pemantauan pertumbuhan

Vercel → Settings → **Spend Management**: pasang batas belanja + notifikasi pada 50/75/100%. Ada opsi "pause project" saat batas tercapai — **jangan aktifkan untuk produksi**: mematikan aplikasi yang sudah dibayar orang tua lebih mahal daripada tagihan kejutan. Notifikasi saja untuk produksi; auto-pause boleh untuk environment beta.

Supabase → Organization → Billing: aktifkan **Spend cap** + usage alert 50/75/90%. Catatan: di Free tier proyek **dijeda setelah ~7 hari tanpa aktivitas** — relevan untuk proyek Supabase beta yang bisa tertidur di antara sesi pengujian.

Pemantauan sendiri (P2, ~2 jam) supaya pertumbuhan terlihat sebelum menjadi tagihan:

```
create or replace function public.ringkas_penyimpanan()
returns json language sql stable security definer set search_path = public as $$
select json_build_object(
  'db_bytes', pg_database_size(current_database()),
  'tabel_terbesar', (
    select coalesce(json_agg(json_build_object(
      'tabel', relname, 'baris', n_live_tup, 'bytes', pg_total_relation_size(relid))
      order by pg_total_relation_size(relid) desc), '[]'::json)
    from (select relname, n_live_tup, relid from pg_stat_user_tables
      where schemaname = 'public'
      order by pg_total_relation_size(relid) desc limit 10) x),
  'storage_per_folder', (
    select coalesce(json_agg(json_build_object('folder', f, 'jml', n, 'bytes', b)), '[]'::json)
    from (select (storage.foldername(name))[1] as f, count(*) n,
      sum((metadata->>'size')::bigint) b
      from storage.objects where bucket_id in ('aset','privat') group by 1) y)
);
$$;
revoke all on function public.ringkas_penyimpanan() from public, anon;
grant execute on function public.ringkas_penyimpanan() to authenticated;
```

Tampilkan di halaman admin bergerbang, atau kirim mingguan lewat cron. Nilainya: pertumbuhan jadi **terlihat** — prasyarat semua keputusan biaya di bab ini.

8. Roadmap & prioritas

8.1 Urutan menjalankan SQL (manual di SQL Editor, berurutan, verifikasi tiap langkah)

#	Berkas	Isi	Prasyarat
1	0000_baseline.sql	Tabel <code>schema_migrations</code> + catat <code>0001 – 0086</code> retroaktif	—
2	0087_perf_index_2.sql	Semua index yang kurang + <code>analyze</code>	0000
3	0087b_ledger_gagal.sql	Outbox kegagalan tulis keuangan	0000
4	0088_rls_initplan.sql	8 policy prioritas — blok demi blok, uji 4 peran tiap blok	0087 (index harus ada dulu)
5	0089_ringkas_rpc.sql	<code>ringkas_keuangan</code> , <code>ringkas_analitik</code> , <code>laporan_engagement</code> versi baru	0087
6	0090_kolom_ringkas.sql	Generated column <code>ada_indikator</code> dll.	Cek jsonb non-array dulu · migrasi dulu, baru deploy kode
7	0091_storage_kebijakan.sql	<code>file_size_limit</code> + <code>allowed_mime_types</code> bucket <code>aset</code>	<code>src/lib/upload.ts</code> sudah ter-deploy
8	0092_bucket_privat.sql	Bucket <code>privat</code> + 4 policy	0091

8.2 Murah & MENDESAK (P0) — ≈ 4–5 hari kerja + \$45/bulan

Kelompok ini menutup ketiga temuan yang sudah berlaku hari ini, plus fondasi yang membuat sisanya bisa dikerjakan dengan aman.

Pekerjaan	Effort	Bab
Rotasi kata sandi admin + audit jejak masuk	1,5 jam	C.5
Cek plafon 1.000 baris (query diagnostik, read-only)	5 menit	D.3.a
Naik Vercel Pro + Supabase Pro (ToS komersial + backup harian)	30 menit / \$45/bln	\$0.3
Kredensial <code>tools/</code> → env var + guard anti-produksi	3–4 jam	C.5
Branch protection <code>master</code> + template PR	45 menit	C.4
Jalankan <code>0087_perf_index_2.sql</code>	15 menit	D.1
Rate limit + delay pada <code>/api/auth/*</code>	2 jam	B.5
<code>.limit()</code> / <code>.range()</code> di 19 route API + deteksi <code>postgrest.terpotong</code>	5–7 jam	B.3
Sentry + <code>instrumentation.ts</code> + <code>error.tsx</code> melaporkan	3 jam	B.2
<code>log.ts</code> + <code>wajib.ts</code> + 10 event wajib	4 jam	B.3
<code>ledger.ts</code> anti-telan + tabel <code>ledger_gagal</code> + alert	4 jam	B.6
<code>/api/health</code> + <code>/api/health/db</code> + uptime monitor + 4 alert PAGE	4 jam	B.1/B.4
<code>0000_baseline.sql</code> + <code>tools/migrate.mjs</code> + gerbang idempotensi di CI	8–10 jam	C.3
<code>backup-db.sh</code> + <code>backup-storage.sh</code> + uji restore pertama	6 jam	C.1
<code>proxy.ts</code> matcher lengkap	15 menit	E.3.c
Aktifkan Web Vitals sebelum mengerjakan Bab E	30 menit	E.5
<code>/pilih-game</code> + <code>admin/tema/[id]</code> → kolom ringan tanpa <code>butir</code>	30 menit	E.2
<code>cacheControl: '31536000'</code> + preset aset 256 px	1,5 jam	F.1
<code>ringkas_keuangan</code> + <code>ringkas_analitik</code> (memperbaiki angka salah)	1,5 hari	D.3
<code>0088</code> RLS blok 1–5 + uji 4 peran	½ hari	D.2
<code>src/lib/upload.ts</code> (satu pintu unggahan)	½ hari	F.1
Bukti/nota baru → bucket privat + <code>urlLampiran()</code> (fase 1)	½ hari	F.3

8.3 Sebelum T1 (~2.000 DAU) — P1

Pekerjaan	Effort	Bab
<code>catatHasilCore</code> berhenti menarik seluruh <code>hasil_main</code> — blocker T1	1–2 hari	A.3 T1
<code>getClaims()</code> menggantikan <code>auth.getUser()</code> di middleware (butuh asymmetric JWT aktif)	3–4 jam	E.3.c
Deploy lewat GitHub Actions: migrate sebelum deploy	3–4 jam	C.4
Code-splitting 15 mesin game (<code>next/dynamic</code>)	3–4 jam	E.1
Caching berlapis + taksonomi tag (3 fase)	2–3 hari	E.3.b
Sampling/batch <code>aktivitas</code> + turunkan 3 index → 1	3 jam	A.3 T1
<code>riwayat_kelas</code> upsert keluar dari jalur GET	2 jam	A.3 T1
<code>ChatKonsultasi</code> polling inkremental + <code>.limit(50)</code>	3 jam	A.3 T1
Environment beta + arahkan <code>tools/</code> ke sana	1–1,5 hari	C.5
Bersihkan data uji di produksi	2–3 jam	C.5
Backfill bukti/nota (fase 2–3) + <code>bersihkan-orphan.mjs</code>	1 hari	F.3/F.4
Sanitasi SVG · tolak GIF · batas PDF · batas bucket	4 jam	F.2/F.5
Retensi <code>aktivitas</code> 180 hari	1 jam	D.3.d
RLS blok 6–8 · <code>Sampul.tsx</code> → <code>next/image</code> · generated column <code>0090</code>	1 hari	D.2/E.2/E.4

8.4 T2 dan seterusnya — P2

Tabel rollup `ringkasan_harian` + cron · arsip/partisi `aktivitas` & `hasil_main` · **PITR** (~\$100/bln) · CDN di depan Storage · rate limit terdistribusi · read replica untuk halaman laporan · Log Drain (retensi > 1 hari) · path ber-entitas + `upsert:true` · Supabase Image Transformation · `~150` policy RLS sisa (hanya bila tabelnya > 1.000 baris) · queue/worker (T3).

8.5 Tiga hal yang harus diverifikasi ulang sebelum menandatangani anggaran 🔍

1. Kuota & harga Vercel Hobby/Pro — Fast Data Transfer, GB-jam, edge request, `maxDuration` default, jumlah cron, apakah rate limit Firewall termasuk di Pro.
2. Kuota & harga Supabase Free/Pro — egress termasuk, ukuran DB, harga tiap tingkat compute, harga PITR, apakah add-on IPv4 diperlukan untuk `pg_dump` .
3. Kuota free tier Sentry dan uptime monitor pilihan.

9. Yang sengaja TIDAK direkomendasikan

Bagian ini sama pentingnya dengan rekomendasinya. Empat hal berikut terlihat seperti perbaikan yang wajar, tapi pada konteks aplikasi ini justru merugikan — dan tanpa catatan ini, kelak ada yang akan "memperbaikinya".

#	Yang tidak dilakukan	Alasan
1	Menaikkan "Max rows" PostgREST untuk memperbaiki angka keuangan yang salah	Hanya memindahkan tebingnya sedikit lebih jauh, sambil menaikkan egress dan memori function. Angka akan salah lagi di ambang baru, tetap tanpa peringatan. Perbaikan yang benar adalah agregasi di SQL (D.3).
2	<code>Aset.tsx</code> → <code>next/image</code>	Menukar egress Supabase yang murah dan berkuota besar dengan kuota Image Optimization Vercel yang sempit, tepat di jalur terpanas dengan ribuan gambar sumber unik. <code>cacheControl</code> + preset 256 px memberi penghematan yang sama tanpa risiko itu (E.4).
3	Library kompresi PDF / optimizer SVG di klien	Puluhan hingga ratusan KB dependency dan CPU berat, ditanggung HP kelas bawah, untuk memperbaiki alur yang hanya dipakai admin . Batas ukuran + instruksi ke admin menyelesaikan 90% masalahnya dengan 0 KB.
4	Membaca isi JWT tanpa memverifikasi tanda tangannya demi menghemat satu perjalanan jaringan	Token yang tidak diverifikasi bisa dipalsukan — di aplikasi ini artinya seseorang bisa mengaku admin. Yang boleh adalah verifikasi lokal berbasis kunci publik (<code>getClaims()</code> + asymmetric JWT). Selama itu belum aktif, <code>getUser()</code> tetap dipakai (E.3.c).
5	Pindah ke VPS/GCP sekarang	Untuk tim satu orang, biaya nyata bukan tagihan bulanan melainkan jam kerja : patching, backup, monitoring, on-call. Selisih biaya baru berarti di atas T3, dan pada titik itu keputusan sebaiknya diambil dengan data operasional yang belum ada hari ini.
6	Menulis ulang 86 migrasi lama jadi idempoten	2–3 hari kerja berisiko tinggi dengan nilai rendah, karena produksi sudah ada. Baseline retroaktif memberi 90% manfaatnya dalam 6–8 jam (C.3).

10. Pemeliharaan dokumen ini

Dokumen ini akan menjadi salah bila tidak dirawat. Tiga aturan:

1. **Angka bertanda 🔍 wajib diverifikasi ulang** sebelum dipakai untuk keputusan anggaran. Yang stabil adalah asumsi & formulanya, bukan nilai kuotanya.
2. **Setiap kali satu butir P0/P1 selesai**, tandai di tabel §8 dengan tanggal — supaya sisa pekerjaan selalu terlihat jelas.
3. **Setiap kali naik tier**, catat metrik nyata (p95 TTFB, CPU Supabase, egress) di tabel di bawah dan bandingkan dengan asumsi A.1. Kalau menyimpang jauh, perbaiki asumsinya — bukan mengabaikan datanya.

Riwayat pengukuran

Tanggal	DAU	p95 TTFB /main	CPU Supabase	Ukuran DB	Egress/bln	Catatan
2026-07-31	—	belum diukur	belum diukur	belum diukur	belum diukur	Audit awal. Web Vitals belum diaktifkan — lihat E.5.

Riwayat uji restore

Backup tanpa uji restore bukan backup. Catat setiap uji, termasuk yang gagal.

Tanggal	Ruang lingkup	Berhasil?	RTO nyata	Temuan
—	belum pernah	—	—	Uji restore pertama masuk P0 (RB-02).

Riwayat insiden

Tanggal	Insiden	Dampak	RPO/RTO nyata	Perubahan pencegah
~2026-07	Kolom <code>kuota_*</code> belum ada saat kode ter-deploy	Daftar event kosong, orang tua gagal mendaftar, admin gagal menyimpan event	—	Pola akses toleran (<code>lib/data/kuota-event.ts</code>). Pencegahan strukturalnya ada di C.4 dan belum dikerjakan.
2026-07-31 s/d 08-05	Deploy berhenti diam-diam ~5 hari. Repo diprivatkan → Vercel plan Hobby memblokir deploy, sementara <code>git push</code> tetap sukses dan CI tetap hijau	6 commit (termasuk 2 perbaikan bug dan 3 fitur) tidak pernah live; terdeteksi hanya karena pemilik melaporkan "fiturnya belum bisa"	Waktu deteksi ~5 hari — tidak ada satu pun sinyal otomatis	Prosedur RB-10 + item digest harian di B.4. Pencegahan permanennya: Vercel Pro agar repo boleh privat tanpa memblokir deploy (\$0.3)